

Project Context: olist-data-warehouse-bi

Project root: /mnt/e/projects/olist-data-warehouse-bi

This PDF is structured for LLM ingestion: file paths are explicit and text blocks are wrapped in code fences.

1. Project Tree

```
olist-data-warehouse-bi/
├── .github/
│   ├── workflows/
│   │   └── ci.yml
│   └── .streamlit/
│       ├── config.toml
│       └── airflow/
│           ├── dags/
│           │   └── olist_etl_dag.py
│           └── dashboard/
│               ├── __init__.py
│               ├── app.py
│               └── README.md
├── data/
├── docs/
│   ├── images/
│   │   ├── dashboard/
│   │   │   ├── dashboard_customer_satisfaction.png
│   │   │   ├── dashboard_delivery_performance.png
│   │   │   ├── dashboard_marketing_funnel.png
│   │   │   ├── dashboard_payment_analysis.png
│   │   │   └── dashboard_sales_overview.png
│   │   └── modelisation_dimensionnelle_olist.png
│   └── logs/
├── .gitkeep
├── sql/
│   ├── 00_create_schemas_and_tables.sql
│   ├── 01_create_marts.sql
│   ├── 99_quality_checks.sql
│   └── olist_postgresql_ddl.sql
├── src/
│   ├── __init__.py
│   ├── config.py
│   ├── db.py
│   ├── extract_load_raw.py
│   ├── load_dwh.py
│   ├── quality_checks.py
│   ├── run_etl.py
│   └── transform_staging.py
├── tests/
│   ├── test_config_paths.py
│   ├── test_dashboard_app.py
│   ├── test_project_structure.py
│   └── test_sql_marts.py
├── .env.example
├── .gitignore
├── docker-compose.yml
├── olist_dw_documentation_youssef_khaloufi.docx
├── olist_dw_documentation_youssef_khaloufi.pdf
├── README.md
└── requirements.txt
```

2. Source And Text Files

.env.example

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/.env.example

```
FILE: .env.example
```text
PostgreSQL connection used by the ETL.
Works with the included docker-compose.yml.
DATABASE_URL=postgresql+psycopg2://olist:olist@localhost:5432/olist_dw

Folder containing the raw Olist CSV files.
RAW_DATA_DIR=data/raw

```

### .gitignore

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/.gitignore

```
FILE: .gitignore
```text
.env
__pycache__/*
*.pyc
.venv/
venv/
logs/*.log
data/raw/*.csv
.ipynb_checkpoints/

---
```

docker-compose.yml

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/docker-compose.yml

```
FILE: docker-compose.yml
```yaml
services:
 postgres:
 image: postgres:16
 container_name: olist_postgres
 environment:
 POSTGRES_DB: olist_dw
 POSTGRES_USER: olist
 POSTGRES_PASSWORD: olist
 ports:
 - "5432:5432"
 volumes:
 - postgres_data:/var/lib/postgresql/data
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U olist -d olist_dw"]
 interval: 5s
 timeout: 5s
 retries: 10

volumes:
 postgres_data:

```

### README.md

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/README.md

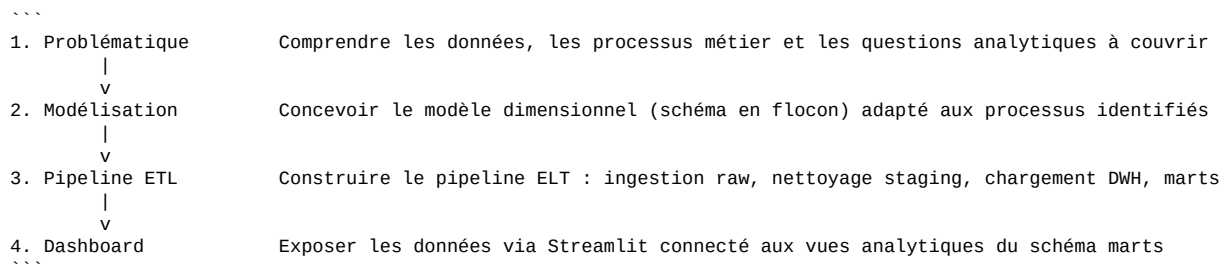
```
FILE: README.md
```markdown
# Olist E-commerce – Data Warehouse & Business Intelligence

> Projet Data Engineering end-to-end sur le dataset public Olist Brazilian E-Commerce.
> Construction d'un pipeline ELT complet : de l'ingestion des données brutes jusqu'à une couche analytique prête pour un da

---
```

Démarche du projet

Ce projet suit une démarche structurée en quatre phases, qui reflète le cycle de vie réel d'un projet Data Engineering en entreprise.



Phase 1 – Problématique

Contexte

Olist est une marketplace brésilienne qui connecte des vendeurs indépendants à plusieurs canaux de vente en ligne. Le dataset est composé de 11 fichiers CSV.

Le point de départ du projet est une question simple : **à partir de 11 fichiers CSV hétérogènes, comment construire une base de données dimensionnelle ?.

Processus métier couverts

L'analyse des sources révèle plusieurs processus distincts, chacun avec sa propre granularité et ses propres dimensions :

Processus	Fichiers sources	Questions analytiques
--- --- ---	--- --- ---	--- --- ---
Ventes et commandes	`orders`, `order_items`	CA par mois, statut, produit, vendeur
Paielements	`order_payments`	Répartition par mode de paiement, nombre de versements
Satisfaction client	`order_reviews`	Distribution des scores, taux d'avis négatifs par catégorie
Logistique	`orders` (timestamps)	Délai moyen, taux de livraison en retard
Catalogue produits	`products`, `category_translation`	Catégories les plus vendues
Géographie	`geolocation`, `customers`, `sellers`	Analyse par ville, État, code postal
Acquisition marketing	`marketing_leads`, `closed_deals`	Conversion leads vers vendeurs par canal

Sources de données

Fichier	Lignes	Description
--- --- ---	--- --- ---	--- --- ---
`olist_orders_dataset.csv`	99 441	Commandes et timestamps du cycle de vie
`olist_order_items_dataset.csv`	112 650	Lignes de commande par produit et vendeur
`olist_order_payments_dataset.csv`	103 886	Paielements (multi-modes par commande)
`olist_order_reviews_dataset.csv`	104 719	Avis clients (score 1-5)
`olist_customers_dataset.csv`	99 441	Clients avec identifiant unique stable
`olist_products_dataset.csv`	32 951	Catalogue produits avec dimensions physiques
`olist_sellers_dataset.csv`	3 095	Vendeurs de la marketplace
`olist_geolocation_dataset.csv`	~1 000 000	Coordonnées GPS par code postal
`product_category_name_translation.csv`	70	Traduction des catégories PT vers EN
`olist_marketing_qualified_leads_dataset.csv`	8 000	Leads marketing top-of-funnel
`olist_closed_deals_dataset.csv`	842	Deals conclus (leads convertis en vendeurs)

Phase 2 – Modélisation dimensionnelle

Choix du schéma en flocon

La diversité des processus métier identifiés en phase 1 impose plusieurs tables de faits indépendantes. Les dimensions géographiques sont regroupées dans une seule table.

Tables de faits

Table	Grain	Indicateurs
--- --- ---	--- --- ---	--- --- ---
`fact_orders`	1 ligne = 1 commande	Délais d'approbation, jours jusqu'à la livraison, retard
`fact_order_items`	1 ligne = 1 article commandé	Prix unitaire, frais de port, valeur totale
`fact_payments`	1 ligne = 1 paiement	Montant, type, nombre de versements
`fact_reviews`	1 ligne = 1 avis client	Score, temps de réponse, indicateurs de sentiment
`fact_marketing_leads`	1 ligne = 1 lead	Canal d'origine, page d'entrée, date de premier contact
`fact_closed_deals`	1 ligne = 1 deal conclu	Vendeur, revenu mensuel déclaré, segment métier

Dimensions

`dim\_date` . `dim\_customer` . `dim\_seller` . `dim\_product` . `dim\_product\_category` . `dim\_order\_status` . `dim\_payment\_type`

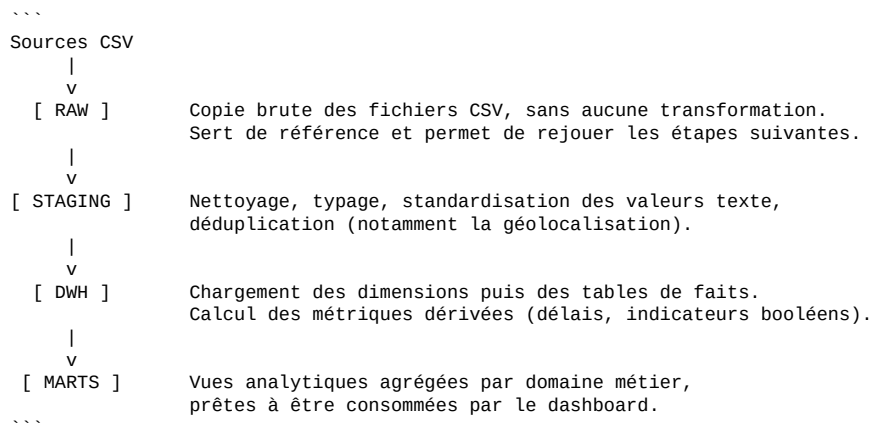
Schéma de modélisation

![Schéma de modélisation dimensionnelle Olist](docs/images/modelisation_dimensionnelle_olist.png)

Phase 3 – Pipeline ETL

Architecture en couches

Le pipeline est organisé en quatre couches PostgreSQL successives, chacune avec un rôle précis :



Structure du projet

```

...
olist-data-warehouse-bi/
■■■■ .github/
■     ■■■■ workflows/
■     ■■■■ ci.yml                # CI GitHub Actions: installation + pytest
■■■■ .streamlit/
■     ■■■■ config.toml          # Configuration visuelle Streamlit
■■■■ airflow/
■     ■■■■ dags/
■     ■■■■ olist_etl_dag.py      # DAG Airflow optionnel (production)
■■■■ dashboard/
■     ■■■■ README.md            # Guide d'utilisation du dashboard
■     ■■■■ app.py               # Dashboard Streamlit connecté aux marts PostgreSQL
■■■■ data/
■     ■■■■ raw/
■     ■■■■ README.md            # Liste des 11 CSV Olist attendus
■■■■ docs/
■     ■■■■ images/
■     ■■■■ modelisation_dimensionnelle_olist.png
■■■■ sql/
■     ■■■■ 00_create_schemas_and_tables.sql # Création des schémas, tables et index
■     ■■■■ 01_create_marts.sql   # Création des vues analytiques marts
■     ■■■■ 99_quality_checks.sql  # Requêtes de diagnostic
■■■■ src/
■     ■■■■ config.py             # Paramètres centralisés (DATABASE_URL, chemins)
■     ■■■■ db.py                # Connexion SQLAlchemy et utilitaires SQL
■     ■■■■ extract_load_raw.py   # Chargement CSV -> raw via COPY FROM STDIN
■     ■■■■ transform_staging.py  # Transformations SQL raw -> staging
■     ■■■■ load_dwh.py           # Chargement staging -> dimensions + faits
■     ■■■■ quality_checks.py     # Contrôles qualité automatisés
■     ■■■■ run_etl.py            # Point d'entrée CLI du pipeline
■■■■ tests/                      # Tests de structure et validation SQL
■■■■ .env.example
■■■■ docker-compose.yml          # PostgreSQL 16 conteneurisé
■■■■ requirements.txt
...

```

Stack technique

Composant	Rôle
Python 3.11+	Orchestration du pipeline, transformations, qualité
PostgreSQL 16	Base de données relationnelle et Data Warehouse
SQLAlchemy 2.x	Couche d'abstraction Python vers PostgreSQL
psycopy2	Chargement performant des CSV via `COPY FROM STDIN`
pandas	Lecture et inspection des fichiers sources
python-dotenv	Configuration par variables d'environnement
Docker Compose	Conteneurisation de PostgreSQL
Apache Airflow	Orchestration du pipeline en production (optionnel)
Streamlit	Interface BI interactive connectée aux vues marts
Plotly	Visualisations interactives du dashboard
Pytest	Validation de structure, routing SQL et vues marts
GitHub Actions	Exécution continue des tests sur push et pull request

Installation et exécution

****Pré-requis**** : Python 3.11+, Docker, fichiers CSV dans `data/raw/`

```

```bash
Cloner le dépôt
git clone https://github.com/<username>/olist-data-warehouse-bi.git

```

```

cd olist-data-warehouse-bi

Environnement Python
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

Configuration
cp .env.example .env

Démarrer PostgreSQL
docker compose up -d

Exécuter le pipeline complet
python -m src.run_etl --step all

Lancer le dashboard
streamlit run dashboard/app.py
```

```

Chaque étape peut également être exécutée individuellement :

```

```bash
python -m src.run_etl --step schema # Création des schémas et tables
python -m src.run_etl --step raw # Chargement des CSV vers raw
python -m src.run_etl --step staging # Transformation vers staging
python -m src.run_etl --step dwh # Chargement vers le Data Warehouse
python -m src.run_etl --step marts # Création des vues analytiques
python -m src.run_etl --step quality # Contrôles qualité
```

```

Résultats du pipeline

****Volume chargé****

| Table | Lignes |
|------------------------|---------|
| `fact_orders` | 99 441 |
| `fact_order_items` | 112 650 |
| `fact_payments` | 103 886 |
| `fact_reviews` | 99 224 |
| `fact_marketing_leads` | 8 000 |
| `fact_closed_deals` | 842 |

****Indicateurs clés****

| Indicateur | Valeur |
|----------------------------|-------------------|
| Valeur totale des articles | 13 591 643,70 R\$ |
| Frais de livraison | 2 251 909,54 R\$ |
| Revenu brut total | 15 843 553,24 R\$ |

****Répartition des paiements****

| Type | Valeur totale |
|-----------------|-------------------|
| Carte de crédit | 12 542 084,19 R\$ |
| Boleto bancário | 2 869 361,27 R\$ |
| Voucher | 379 436,87 R\$ |
| Carte de débit | 217 989,79 R\$ |

****Distribution des avis clients****

| Score | Nombre d'avis | Part |
|-------|---------------|--------|
| 5 | 57 328 | 57,8 % |
| 4 | 19 142 | 19,3 % |
| 3 | 8 179 | 8,2 % |
| 2 | 3 151 | 3,2 % |
| 1 | 11 424 | 11,5 % |

Contrôles qualité

8 contrôles automatisés sont exécutés après le chargement du Data Warehouse. ****Résultat : 8 / 8 contrôles réussis.****

| Contrôle | Résultat |
|---|----------|
| Données commandes chargées dans `raw` | PASS |
| `fact_orders` non vide | PASS |
| `fact_order_items` – aucun `order_key` manquant | PASS |
| `fact_order_items` – aucun `product_key` manquant | PASS |
| `fact_order_items` – aucun `seller_key` manquant | PASS |
| `fact_payments` – aucun `order_key` manquant | PASS |
| `fact_reviews` – aucun `order_key` manquant | PASS |
| Scores d'avis compris entre 1 et 5 | PASS |

Phase 4 – Dashboard Streamlit

Le dashboard Streamlit est implémenté et connecté directement aux vues analytiques PostgreSQL du schéma `marts`. Il consomme

Pages du dashboard

```
Page	Vues marts utilisées	Contenu
Vue globale des ventes	`marts.sales_overview`, `marts.sales_by_category`	Revenu mensuel, volumes de commandes, statut
Analyse des paiements	`marts.payment_analysis`	Répartition par mode de paiement, valeur totale et distribution des ve
Satisfaction client	`marts.customer_satisfaction`	Score moyen, distribution des avis et parts d'avis positifs / négat
Performance logistique	`marts.delivery_performance`	Délai moyen, écart avec la date estimée et taux de livraison à te
Tunnel marketing	`marts.marketing_funnel`	MQL, deals gagnés, revenu déclaré et conversion par canal
Contrôles qualité	tables `raw`, `dwh`, `marts`	Vérifications rapides de cohérence et disponibilité des données
```

Le dashboard affiche également la cible PostgreSQL active (hôte / base) sans exposer le mot de passe, et gère les jeux de d

TODO: ajouter de vraies captures du dashboard une fois des exports validés seront disponibles.

Etat d'avancement

```
Phase	Etape	Statut
Problématique	Analyse des sources et documentation	Termine
Modélisation	Schéma dimensionnel en flocon	Termine
ETL	Création des schémas PostgreSQL	Termine
ETL	Chargement couche raw	Termine
ETL	Transformation couche staging	Termine
ETL	Chargement Data Warehouse	Termine
ETL	Création des marts analytiques	Termine
ETL	Contrôles qualité	Termine
Dashboard	Développement Streamlit	Terminé
CI	Tests de structure et validation SQL	Terminé
```

Auteur

****YOUSSEF KHALOUFI****

Data Engineering Student

[LinkedIn](https://linkedin.com/in/khaloufi-youssef) · [GitHub](https://github.com/KYoussefAI/olist-data-warehouse-bi)

Licence

Ce projet est open-source sous licence [MIT](LICENSE).

Les datasets utilisés sont disponibles publiquement sur Kaggle sous licence CC BY-NC-SA 4.0 :

- [Olist Brazilian E-Commerce](https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce)
- [Olist Marketing Funnel](https://www.kaggle.com/datasets/olistbr/marketing-funnel-olist)

...

requirements.txt

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/requirements.txt

```
FILE: requirements.txt
```text
pandas>=2.0.0
SQLAlchemy>=2.0.0
psycopg2-binary>=2.9.0
python-dotenv>=1.0.0
streamlit>=1.35.0
plotly>=5.20.0
pytest>=8.0.0
```
```

.github/workflows/ci.yml

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/.github/workflows/ci.yml

```
FILE: .github/workflows/ci.yml
```yaml
name: CI
```

```

on:
 push:
 branches: [main, master]
 pull_request:
 branches: [main, master]

jobs:
 test:
 runs-on: ubuntu-latest

 steps:
 - name: Checkout
 uses: actions/checkout@v4

 - name: Set up Python
 uses: actions/setup-python@v5
 with:
 python-version: "3.11"

 - name: Install dependencies
 run: pip install -r requirements.txt

 - name: Run tests
 run: pytest -q

```

## **.streamlit/config.toml**

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/.streamlit/config.toml

```

FILE: .streamlit/config.toml
```toml
[theme]
primaryColor = "#0F766E"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F8FAFC"
textColor = "#0F172A"
font = "sans serif"

[server]
headless = true

```

airflow/dags/olist_etl_dag.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/airflow/dags/olist_etl_dag.py

```

FILE: airflow/dags/olist_etl_dag.py
```python
"""Optional Apache Airflow DAG for the Olist ETL pipeline.

This DAG assumes the project is available inside the Airflow worker and that
DATABASE_URL / RAW_DATA_DIR are configured as environment variables.
"""

from __future__ import annotations

from datetime import datetime

from airflow import DAG
from airflow.operators.bash import BashOperator

with DAG(
 dag_id="olist_postgres_elt_pipeline",
 start_date=datetime(2024, 1, 1),
 schedule=None,
 catchup=False,
 tags=["olist", "postgres", "elt", "portfolio"],
) as dag:
 create_schema = BashOperator(
 task_id="create_schema",
 bash_command="python -m src.run_etl --step schema",
)

 load_raw = BashOperator(
 task_id="load_raw_csvs",
 bash_command="python -m src.run_etl --step raw",
)

```

```

build_staging = BashOperator(
 task_id="build_staging",
 bash_command="python -m src.run_etl --step staging",
)

load_dwh = BashOperator(
 task_id="load_dwh",
 bash_command="python -m src.run_etl --step dwh",
)

build_marts = BashOperator(
 task_id="build_marts",
 bash_command="python -m src.run_etl --step marts",
)

run_quality = BashOperator(
 task_id="run_quality_checks",
 bash_command="python -m src.run_etl --step quality --fail-fast-quality",
)

create_schema >> load_raw >> build_staging >> load_dwh >> build_marts >> run_quality
...

```

## dashboard/\_\_init\_\_.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/dashboard/\_\_init\_\_.py

```

FILE: dashboard/__init__.py
```python
"""Streamlit dashboard package for the Olist BI project."""
...

```

dashboard/app.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/dashboard/app.py

```

FILE: dashboard/app.py
```python
from __future__ import annotations

import os
from typing import Iterable

import pandas as pd
import plotly.express as px
import streamlit as st
from dotenv import load_dotenv
from sqlalchemy import create_engine, text
from sqlalchemy.engine import Engine
from sqlalchemy.engine.url import make_url
from sqlalchemy.exc import SQLAlchemyError

load_dotenv()

DATABASE_URL = os.getenv(
 "DATABASE_URL",
 "postgresql+psycopg2://olist:olist@localhost:5432/olist_dw",
)

PAGE_CONFIG = {
 "Executive · Ventes": {
 "title": "Vue globale des ventes",
 "subtitle": "Suivi executive des revenus, commandes, statuts et categories prioritaires.",
 "handler": "page_sales",
 },
 "Commerce · Paiements": {
 "title": "Analyse des paiements",
 "subtitle": "Lecture business des montants, modes de paiement et comportements de versement.",
 "handler": "page_payments",
 },
 "Commerce · Satisfaction": {
 "title": "Satisfaction client",
 "subtitle": "Synthese des avis clients pour suivre la perception et les signaux de friction.",
 "handler": "page_satisfaction",
 },
 "Operations · Logistique": {
 "title": "Performance logistique",
 "subtitle": "Pilotage des delais de livraison, des ecarts a la promesse et du taux a temps.",
 "handler": "page_delivery",
 },
}

```

```

 },
 "Growth · Marketing": {
 "title": "Tunnel marketing",
 "subtitle": "Vue funnel des leads qualifiées, deals signés et revenus déclarés par origine.",
 "handler": "page_marketing",
 },
 "Data Quality · Contrôles": {
 "title": "Contrôles qualité",
 "subtitle": "Vérification des chargements critiques et des contrôles de cohérence marts/DWH.",
 "handler": "page_quality",
 },
 },
}

NAV_OPTIONS = {
 "Executive · Ventes": "sales",
 "Commerce · Paiements": "payments",
 "Commerce · Satisfaction": "satisfaction",
 "Operations · Logistique": "delivery",
 "Growth · Marketing": "marketing",
 "Data Quality · Contrôles": "quality",
}

PLOTLY_TEMPLATE = "plotly_white"
CHART_HEIGHT = 340
ACCENT = "#0F766E"
ACCENT_LIGHT = "#CCFBF1"
TEXT_MUTED = "#64748B"
BORDER = "#E2E8F0"
SURFACE = "#FFFFFF"
SURFACE_ALT = "#F8FAFC"

st.set_page_config(
 page_title="Olist BI Dashboard",
 page_icon="■",
 layout="wide",
 initial_sidebar_state="expanded",
)

st.markdown(
 f"""
<style>
.stApp {{
 background: linear-gradient(180deg, #F8FAFC 0%, #EEF4F7 100%);
}}
.block-container {{
 padding-top: 1.2rem;
 padding-bottom: 2rem;
 max-width: 1380px;
}}
[data-testid="stSidebar"] {{
 background: #F8FAFC;
 border-right: 1px solid {BORDER};
}}
.sidebar-card {{
 background: {SURFACE};
 border: 1px solid {BORDER};
 border-radius: 14px;
 padding: 0.9rem 1rem;
 margin: 0.25rem 0 1rem 0;
}}
.page-header {{
 background: {SURFACE};
 border: 1px solid {BORDER};
 border-radius: 18px;
 padding: 1.25rem 1.35rem;
 margin-bottom: 1.1rem;
 box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04);
}}
.page-header-top {{
 display: flex;
 justify-content: space-between;
 gap: 1rem;
 align-items: flex-start;
}}
.page-title {{
 margin: 0;
 font-size: 1.8rem;
 font-weight: 700;
 color: #0F172A;
}}
.page-subtitle {{
 margin: 0.45rem 0 0 0;
 color: {TEXT_MUTED};
 font-size: 0.97rem;
 line-height: 1.5;
}}
.db-badge {{

```

```

 display: inline-flex;
 align-items: center;
 white-space: nowrap;
 background: {SURFACE_ALT};
 border: 1px solid {BORDER};
 border-radius: 999px;
 padding: 0.35rem 0.75rem;
 color: #0F172A;
 font-size: 0.82rem;
 font-weight: 600;
 }}
 .section-title {{
 margin: 1.15rem 0 0.35rem 0;
 font-size: 1.02rem;
 font-weight: 700;
 color: #0F172A;
 }}
 .section-description {{
 margin: 0 0 0.8rem 0;
 color: {TEXT_MUTED};
 font-size: 0.9rem;
 }}
 .metric-card {{
 background: {SURFACE};
 border: 1px solid {BORDER};
 border-radius: 16px;
 padding: 1rem 1.05rem;
 min-height: 122px;
 box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04);
 }}
 .metric-label {{
 color: {TEXT_MUTED};
 font-size: 0.84rem;
 font-weight: 600;
 margin-bottom: 0.55rem;
 text-transform: uppercase;
 letter-spacing: 0.03em;
 }}
 .metric-value {{
 color: #0F172A;
 font-size: 1.7rem;
 font-weight: 700;
 line-height: 1.15;
 }}
 .metric-help {{
 margin-top: 0.6rem;
 color: {TEXT_MUTED};
 font-size: 0.82rem;
 }}
 .panel-card {{
 background: {SURFACE};
 border: 1px solid {BORDER};
 border-radius: 18px;
 padding: 0.9rem 1rem 0.35rem 1rem;
 box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04);
 }}
 .small-caption {{
 color: {TEXT_MUTED};
 font-size: 0.82rem;
 }}
 .quality-table {{
 width: 100%;
 border-collapse: collapse;
 font-size: 0.92rem;
 }}
 .quality-table th {{
 text-align: left;
 color: {TEXT_MUTED};
 font-weight: 600;
 padding: 0.75rem 0.65rem;
 border-bottom: 1px solid {BORDER};
 }}
 .quality-table td {{
 padding: 0.8rem 0.65rem;
 border-bottom: 1px solid #F1F5F9;
 color: #0F172A;
 }}
 .status-badge {{
 display: inline-block;
 border-radius: 999px;
 padding: 0.18rem 0.58rem;
 font-size: 0.78rem;
 font-weight: 700;
 }}
 .status-pass {{
 background: #DCFCE7;
 color: #166534;
 }}

```

```

 }}
 .status-fail {{
 background: #FEE2E2;
 color: #991B1B;
 }}
 div[data-testid="stExpander"] {{
 border: 1px solid {BORDER};
 border-radius: 16px;
 background: {SURFACE};
 }}
</style>
""",
 unsafe_allow_html=True,
)

@st.cache_resource(show_spinner=False)
def get_engine() -> Engine:
 return create_engine(DATABASE_URL, pool_pre_ping=True, future=True)

@st.cache_data(ttl=300, show_spinner=False)
def run_query(sql: str, params: dict | None = None) -> pd.DataFrame:
 engine = get_engine()
 with engine.connect() as conn:
 return pd.read_sql_query(text(sql), conn, params=params or {})

def safe_query(sql: str, params: dict | None = None) -> pd.DataFrame:
 try:
 return run_query(sql, params)
 except SQLAlchemyError as exc:
 st.error(
 "Impossible de lire les donnees PostgreSQL. Verifiez que PostgreSQL est accessible et que les vues marts ont ete creees."
)
 with st.expander("Detail technique"):
 st.code(str(exc))
 return pd.DataFrame()

def format_brl(value: float | int | None) -> str:
 if value is None or pd.isna(value):
 return "R$ 0"
 return f"R$ {value:,.0f}".replace(",", " ")

def format_pct(value: float | int | None) -> str:
 if value is None or pd.isna(value):
 return "0.0 %"
 return f"{value * 100:.1f} %"

def format_count(value: float | int | None) -> str:
 if value is None or pd.isna(value):
 return "0"
 return f"{int(value):,}".replace(",", " ")

def database_target() -> str:
 try:
 url = make_url(DATABASE_URL)
 host = url.host or "localhost"
 database = url.database or "unknown"
 return f"{host} / {database}"
 except Exception:
 return "configuration DATABASE_URL non lisible"

def as_period(df: pd.DataFrame) -> pd.DataFrame:
 if df.empty or not {"year_number", "month_number"}.issubset(df.columns):
 return df
 out = df.copy()
 out["period"] = pd.to_datetime(
 out["year_number"].astype(int).astype(str)
 + "-"
 + out["month_number"].astype(int).astype(str)
 + "-01"
)
 return out.sort_values("period")

def year_filter_clause(alias: str = "") -> str:
 column = f"{alias}.year_number" if alias else "year_number"
 return f"WHERE (:use_year_filter = FALSE OR {column} = ANY(:years))"

def params() -> dict:

```

```

selected_years = st.session_state.get("selected_years") or []
return {
 "years": selected_years,
 "use_year_filter": bool(selected_years),
}

def load_available_years() -> list[int]:
 df = safe_query(
 """
 SELECT DISTINCT year_number
 FROM marts.sales_overview
 WHERE year_number IS NOT NULL
 ORDER BY year_number
 """
)
 if df.empty:
 return []
 return [int(x) for x in df["year_number"].dropna().tolist()]

def page_header(title: str, subtitle: str) -> None:
 st.markdown(
 f"""
 <div class="page-header">
 <div class="page-header-top">
 <div>
 <h1 class="page-title">{title}</h1>
 <p class="page-subtitle">{subtitle}</p>
 </div>
 <div class="db-badge">Base active: {database_target()}</div>
 </div>
 </div>
 """,
 unsafe_allow_html=True,
)

def metric_card(label: str, value: str, help_text: str | None = None) -> None:
 help_block = f'<div class="metric-help">{help_text}</div>' if help_text else ""
 st.markdown(
 f"""
 <div class="metric-card">
 <div class="metric-label">{label}</div>
 <div class="metric-value">{value}</div>
 {help_block}
 </div>
 """,
 unsafe_allow_html=True,
)

def metric_grid(metrics: Iterable[tuple[str, str, str | None]]) -> None:
 metrics = list(metrics)
 cols = st.columns(len(metrics))
 for col, (label, value, help_text) in zip(cols, metrics):
 with col:
 metric_card(label, value, help_text)

def section_title(title: str, description: str | None = None) -> None:
 st.markdown(f'<div class="section-title">{title}</div>', unsafe_allow_html=True)
 if description:
 st.markdown(
 f'<div class="section-description">{description}</div>',
 unsafe_allow_html=True,
)

def standard_chart_layout(fig, height: int = CHART_HEIGHT):
 fig.update_layout(
 template=PLOTLY_TEMPLATE,
 height=height,
 paper_bgcolor="rgba(0,0,0,0)",
 plot_bgcolor="rgba(0,0,0,0)",
 margin=dict(l=20, r=20, t=50, b=20),
 font=dict(color="#0F172A"),
 title=dict(font=dict(size=16)),
 legend=dict(
 orientation="h",
 yanchor="bottom",
 y=1.02,
 xanchor="right",
 x=1,
 title_text="",
),
 xaxis=dict(showgrid=True, gridcolor="#E2E8F0", zeroline=False),
)

```

```

 yaxis=dict(showgrid=True, gridcolor="#E2E8F0", zeroline=False),
)
 return fig

def chart_panel(fig) -> None:
 with st.container():
 st.markdown('<div class="panel-card">', unsafe_allow_html=True)
 st.plotly_chart(fig, use_container_width=True)
 st.markdown("</div>", unsafe_allow_html=True)

def show_empty_state(subject: str) -> None:
 st.warning(
 f"Aucune donnee disponible pour {subject}. Verifiez que l'ETL et la creation des vues marts ont bien ete executes."
)

def render_quality_table(checks: pd.DataFrame) -> None:
 rows: list[str] = []
 for row in checks.itertuples(index=False):
 badge_class = "status-pass" if row.passed else "status-fail"
 badge_text = "PASS" if row.passed else "FAIL"
 rows.append(
 f"""
 <tr>
 <td>{badge_text}</td>
 <td>{row.check_name}</td>
 <td>{row.value}</td>
 </tr>
 """
)

 st.markdown(
 """
 <div class="panel-card">
 <table class="quality-table">
 <thead>
 <tr>
 <th>Statut</th>
 <th>Controle</th>
 <th>Valeur</th>
 </tr>
 </thead>
 <tbody>
 """
 + "".join(rows)
 + """
 </tbody>
 </table>
 </div>
 """,
 unsafe_allow_html=True,
)

def sidebar() -> str:
 st.sidebar.markdown("## Olist BI Dashboard")
 st.sidebar.markdown(
 '<div class="small-caption">Data Warehouse Analytics</div>',
 unsafe_allow_html=True,
)
 st.sidebar.markdown(
 f"""
 <div class="sidebar-card">
 <div class="metric-label">Base active</div>
 <div style="font-weight:700;color:#0F172A;">{database_target()}</div>
 </div>
 """,
 unsafe_allow_html=True,
)

 st.sidebar.markdown("### Filtres globaux")
 years = load_available_years()
 selected_years = st.sidebar.multiselect(
 "Annees",
 options=years,
 default=years,
 help="Filtre global applique aux pages temporelles.",
)
 st.session_state["selected_years"] = selected_years

 st.sidebar.markdown("### Navigation")
 selected_label = st.sidebar.selectbox("Navigation", list(NAV_OPTIONS.keys()))
 st.sidebar.caption(f"Page active : {selected_label}")
 st.sidebar.markdown("---")
 st.sidebar.markdown(

```

```

 '<div class="small-caption">Run: streamlit run dashboard/app.py</div>',
 unsafe_allow_html=True,
)
 return NAV_OPTIONS[selected_label]

def page_sales() -> None:
 config = PAGE_CONFIG["Executive · Ventres"]
 page_header(config["title"], config["subtitle"])

 df = safe_query(
 """
 SELECT
 year_number,
 month_number,
 month_name,
 order_status,
 order_count,
 item_count,
 gross_revenue
 FROM marts.sales_overview
 {year_filter_clause()}
 ORDER BY year_number, month_number, order_status
 """
 ,
 params(),
)
 df = as_period(df)
 if df.empty:
 show_empty_state("la vue globale des ventes")
 return

 total_revenue = float(df["gross_revenue"].fillna(0).sum())
 total_orders = int(df["order_count"].fillna(0).sum())
 total_items = int(df["item_count"].fillna(0).sum())
 avg_order_value = total_revenue / total_orders if total_orders else 0

 section_title("Indicateurs cles")
 metric_grid(
 [
 ("Revenu brut", format_brl(total_revenue), "Somme du revenu brut observe dans marts.sales_overview."),
 ("Commandes", format_count(total_orders), "Volume total de commandes sur la selection active."),
 ("Articles vendus", format_count(total_items), "Nombre d'articles vendus dans les commandes filtrees."),
 ("Panier moyen", format_brl(avg_order_value), "Revenu brut moyen par commande."),
]
)

 monthly = df.groupby("period", as_index=False).agg(
 gross_revenue=("gross_revenue", "sum"),
 order_count=("order_count", "sum"),
 item_count=("item_count", "sum"),
)
 status = df.groupby("order_status", as_index=False).agg(order_count=("order_count", "sum"))
 status["share"] = status["order_count"] / status["order_count"].sum()
 status = status.sort_values("order_count", ascending=True)

 section_title("Evolution temporelle", "Lecture mensuelle des revenus generes.")
 left, right = st.columns([1.7, 1])
 with left:
 revenue_fig = px.line(
 monthly,
 x="period",
 y="gross_revenue",
 markers=True,
 title="Revenu mensuel",
 color_discrete_sequence=[ACCENT],
)
 revenue_fig.update_traces(line_width=3)
 revenue_fig.update_layout(xaxis_title="Mois", yaxis_title="Revenu brut")
 chart_panel(standard_chart_layout(revenue_fig))
 with right:
 section_title("Repartition", "Distribution des commandes par statut.")
 status_fig = px.bar(
 status,
 x="order_count",
 y="order_status",
 orientation="h",
 title="Commandes par statut",
 text="order_count",
 color_discrete_sequence=[ACCENT],
 hover_data={"share": ":.1%"},
)
 status_fig.update_layout(xaxis_title="Commandes", yaxis_title="Statut")
 chart_panel(standard_chart_layout(status_fig))

 section_title("Top categories", "Top 15 categories produit par revenu brut.")
 cat = safe_query(
 """

```

```

 SELECT product_category, item_count, order_count, gross_revenue
 FROM marts.sales_by_category
 ORDER BY gross_revenue DESC
 LIMIT 15
 """
)
 if cat.empty:
 show_empty_state("les categories produits")
 else:
 cat_fig = px.bar(
 cat.sort_values("gross_revenue", ascending=True),
 x="gross_revenue",
 y="product_category",
 orientation="h",
 text="gross_revenue",
 title="Top categories par revenu brut",
 color_discrete_sequence=[ACCENT],
)
 cat_fig.update_layout(xaxis_title="Revenu brut", yaxis_title="Categorie")
 chart_panel(standard_chart_layout(cat_fig, height=400))

 section_title("Details")
 with st.expander("Voir les donnees agregees"):
 st.dataframe(monthly, use_container_width=True, hide_index=True)

def page_payments() -> None:
 config = PAGE_CONFIG["Commerce · Paiements"]
 page_header(config["title"], config["subtitle"])

 df = safe_query(
 f"""
 SELECT
 year_number,
 month_number,
 month_name,
 payment_type,
 payment_installments,
 payment_count,
 total_payment_value
 FROM marts.payment_analysis
 {year_filter_clause()}
 ORDER BY year_number, month_number, payment_type, payment_installments
 """
 ,
 params(),
)
 df = as_period(df)
 if df.empty:
 show_empty_state("l'analyse des paiements")
 return

 total_value = float(df["total_payment_value"].fillna(0).sum())
 total_count = int(df["payment_count"].fillna(0).sum())
 avg_payment = total_value / total_count if total_count else 0
 installments_avg = (
 float(
 (df["payment_installments"].fillna(0) * df["payment_count"].fillna(0)).sum()
 / total_count
)
 if total_count
 else 0
)

 section_title("Indicateurs cles")
 metric_grid(
 [
 ("Valeur totale payee", format_brl(total_value), None),
 ("Nombre de paiements", format_count(total_count), None),
 ("Paiement moyen", format_brl(avg_payment), None),
 ("Versements moyens", f"{installments_avg:.2f}", None),
]
)

 by_type = df.groupby("payment_type", as_index=False).agg(
 total_payment_value=("total_payment_value", "sum"),
 payment_count=("payment_count", "sum"),
).sort_values("total_payment_value", ascending=False)

 section_title("Repartition", "Lecture comparee des montants et volumes par mode de paiement.")
 left, right = st.columns(2)
 with left:
 value_fig = px.bar(
 by_type,
 x="payment_type",
 y="total_payment_value",
 title="Valeur par mode de paiement",
 color_discrete_sequence=[ACCENT],

```

```

)
 value_fig.update_layout(xaxis_title="Mode", yaxis_title="Valeur totale")
 chart_panel(standard_chart_layout(value_fig))
with right:
 count_fig = px.bar(
 by_type,
 x="payment_type",
 y="payment_count",
 title="Nombre de paiements par mode",
 color_discrete_sequence=["#0F172A"],
)
 count_fig.update_layout(xaxis_title="Mode", yaxis_title="Nombre de paiements")
 chart_panel(standard_chart_layout(count_fig))

section_title("Evolution temporelle", "Distribution des versements sur le perimetre selectionne.")
installments = df.groupby("payment_installments", as_index=False).agg(
 payment_count=("payment_count", "sum")
)
installments_fig = px.bar(
 installments,
 x="payment_installments",
 y="payment_count",
 title="Distribution des versements",
 color_discrete_sequence=[ACCENT],
)
installments_fig.update_layout(
 xaxis_title="Nombre de versements",
 yaxis_title="Nombre de paiements",
)
chart_panel(standard_chart_layout(installments_fig))

def page_satisfaction() -> None:
 config = PAGE_CONFIG["Commerce · Satisfaction"]
 page_header(config["title"], config["subtitle"])

 df = safe_query(
 f"""
 SELECT
 year_number,
 month_number,
 month_name,
 review_score,
 review_count
 FROM marts.customer_satisfaction
 {year_filter_clause()}
 ORDER BY year_number, month_number, review_score
 """,
 params(),
)
 df = as_period(df)
 if df.empty:
 show_empty_state("la satisfaction client")
 return

 total_reviews = int(df["review_count"].fillna(0).sum())
 weighted_score = (
 (df["review_score"] * df["review_count"]).sum() / total_reviews if total_reviews else 0
)
 negative = int(df.loc[df["review_score"].isin([1, 2]), "review_count"].fillna(0).sum())
 positive = int(df.loc[df["review_score"].isin([4, 5]), "review_count"].fillna(0).sum())

 section_title("Indicateurs cles")
 metric_grid(
 [
 ("Nombre d'avis", format_count(total_reviews), None),
 ("Score moyen", f"{weighted_score:.2f} / 5", None),
 ("Avis positifs", format_pct(positive / total_reviews if total_reviews else 0), None),
 ("Avis negatifs", format_pct(negative / total_reviews if total_reviews else 0), None),
]
)
 st.caption("Les scores 4-5 sont consideres comme positifs, 1-2 comme negatifs.")

 monthly = (
 df.groupby("period", as_index=False)
 .apply(
 lambda group: pd.Series(
 {
 "avg_score": (
 (group["review_score"] * group["review_count"]).sum()
 / group["review_count"].sum()
),
 "review_count": group["review_count"].sum(),
 }
)
)
 .reset_index(drop=True)

```

```

)
distribution = df.groupby("review_score", as_index=False).agg(
 review_count=("review_count", "sum")
)

section_title("Evolution temporelle", "Evolution mensuelle du score moyen.")
left, right = st.columns([1.4, 1])
with left:
 monthly_fig = px.line(
 monthly,
 x="period",
 y="avg_score",
 markers=True,
 title="Score moyen mensuel",
 color_discrete_sequence=[ACCENT],
)
 monthly_fig.update_layout(xaxis_title="Mois", yaxis_title="Score moyen")
 monthly_fig.update_yaxes(range=[0, 5])
 chart_panel(standard_chart_layout(monthly_fig))
with right:
 section_title("Repartition", "Distribution des avis par score.")
 distribution_fig = px.bar(
 distribution,
 x="review_score",
 y="review_count",
 title="Distribution des scores",
 color_discrete_sequence=["#0F172A"],
)
 distribution_fig.update_layout(xaxis_title="Score", yaxis_title="Nombre d'avis")
 chart_panel(standard_chart_layout(distribution_fig))

def page_delivery() -> None:
 config = PAGE_CONFIG["Operations · Logistique"]
 page_header(config["title"], config["subtitle"])

 df = safe_query(
 f"""
 SELECT
 year_number,
 month_number,
 month_name,
 order_status,
 delivered_orders,
 avg_days_purchase_to_delivery,
 avg_days_vs_estimate,
 on_time_rate
 FROM marts.delivery_performance
 {year_filter_clause()}
 ORDER BY year_number, month_number, order_status
 """,
 params(),
)
 df = as_period(df)
 if df.empty:
 show_empty_state("la performance logistique")
 return

 total_delivered = int(df["delivered_orders"].fillna(0).sum())
 avg_delivery = (
 float(
 (df["avg_days_purchase_to_delivery"].fillna(0) * df["delivered_orders"].fillna(0)).sum()
 / total_delivered
)
 if total_delivered
 else 0
)
 avg_vs_estimate = (
 float(
 (df["avg_days_vs_estimate"].fillna(0) * df["delivered_orders"].fillna(0)).sum()
 / total_delivered
)
 if total_delivered
 else 0
)
 on_time_rate = (
 float((df["on_time_rate"].fillna(0) * df["delivered_orders"].fillna(0)).sum() / total_delivered)
 if total_delivered
 else 0
)

 section_title("Indicateurs cles")
 metric_grid(
 [
 ("Commandes livrees", format_count(total_delivered), None),
 ("Delai moyen", f"{avg_delivery:.1f} jours", None),
 ("Ecart vs estimation", f"{avg_vs_estimate:.1f} jours", None),
]
)

```

```

 ("Taux a temps", format_pct(on_time_rate), None),
]
)

monthly = (
 df.groupby("period", as_index=False)
 .apply(
 lambda group: pd.Series(
 {
 "avg_days_purchase_to_delivery": (
 (group["avg_days_purchase_to_delivery"].fillna(0) * group["delivered_orders"]).sum()
 / group["delivered_orders"].sum()
),
 "on_time_rate": (
 (group["on_time_rate"].fillna(0) * group["delivered_orders"]).sum()
 / group["delivered_orders"].sum()
),
 }
)
)
 .reset_index(drop=True)
)

by_status = df.groupby("order_status", as_index=False).agg(
 on_time_rate=(
 "on_time_rate",
 lambda series: (
 (
 series.fillna(0)
 * df.loc[series.index, "delivered_orders"].fillna(0)
).sum()
 / df.loc[series.index, "delivered_orders"].fillna(0).sum()
)
 if df.loc[series.index, "delivered_orders"].fillna(0).sum()
 else 0
)
)

by_status = by_status.sort_values("on_time_rate", ascending=True)

section_title("Evolution temporelle", "Suivi mensuel des delais et du respect de la promesse client.")
left, right = st.columns(2)
with left:
 delay_fig = px.line(
 monthly,
 x="period",
 y="avg_days_purchase_to_delivery",
 markers=True,
 title="Delai moyen de livraison",
 color_discrete_sequence=ACCENT,
)
 delay_fig.update_layout(xaxis_title="Mois", yaxis_title="Jours")
 chart_panel(standard_chart_layout(delay_fig))
with right:
 on_time_fig = px.bar(
 by_status,
 x="on_time_rate",
 y="order_status",
 orientation="h",
 title="Taux a temps par statut",
 text="on_time_rate",
 color_discrete_sequence=ACCENT,
)
 on_time_fig.update_traces(texttemplate="%{text:.0%}")
 on_time_fig.update_layout(xaxis_title="Taux a temps", yaxis_title="Statut")
 chart_panel(standard_chart_layout(on_time_fig))

def page_marketing() -> None:
 config = PAGE_CONFIG["Growth · Marketing"]
 page_header(config["title"], config["subtitle"])

 df = safe_query(
 """
 SELECT
 year_number,
 month_number,
 month_name,
 marketing_origin,
 mql_count,
 won_deal_count,
 declared_monthly_revenue_sum,
 conversion_rate
 FROM marts.marketing_funnel
 {year_filter_clause()}
 ORDER BY year_number, month_number, marketing_origin
 """,
 params(),
)

```

```

df = as_period(df)
if df.empty:
 show_empty_state("le tunnel marketing")
 return

total_leads = int(df["mql_count"].fillna(0).sum())
total_deals = int(df["won_deal_count"].fillna(0).sum())
revenue = float(df["declared_monthly_revenue_sum"].fillna(0).sum())
conversion = total_deals / total_leads if total_leads else 0

section_title("Indicateurs clés")
metric_grid(
 [
 ("MQL", format_count(total_leads), None),
 ("Deals gagnés", format_count(total_deals), None),
 ("Conversion", format_pct(conversion), None),
 ("Revenu déclaré", format_brl(revenue), None),
]
)

by_origin = df.groupby("marketing_origin", as_index=False).agg(
 mql_count=("mql_count", "sum"),
 won_deal_count=("won_deal_count", "sum"),
 declared_monthly_revenue_sum=("declared_monthly_revenue_sum", "sum"),
)
by_origin["conversion_rate"] = (
 by_origin["won_deal_count"] / by_origin["mql_count"].replace(0, pd.NA)
)
by_origin = by_origin.sort_values("mql_count", ascending=False).head(15)

section_title("Repartition", "Comparaison du haut et du bas de funnel par origine marketing.")
left, right = st.columns(2)
with left:
 volume_fig = px.bar(
 by_origin,
 x="marketing_origin",
 y=["mql_count", "won_deal_count"],
 barmode="group",
 title="MQL vs deals par origine",
 color_discrete_sequence=[ACCENT, "#0F172A"],
)
 volume_fig.update_layout(xaxis_title="Origine", yaxis_title="Volume")
 chart_panel(standard_chart_layout(volume_fig))
with right:
 conversion_fig = px.bar(
 by_origin.sort_values("conversion_rate", ascending=False),
 x="marketing_origin",
 y="conversion_rate",
 title="Taux de conversion par origine",
 color_discrete_sequence=[ACCENT],
 text="conversion_rate",
)
 conversion_fig.update_traces(texttemplate="%{text:.0%}")
 conversion_fig.update_layout(xaxis_title="Origine", yaxis_title="Conversion")
 chart_panel(standard_chart_layout(conversion_fig))

section_title("Details")
with st.expander("Voir les principales origines marketing"):
 display_df = by_origin.copy()
 display_df["conversion_rate"] = display_df["conversion_rate"].map(format_pct)
 display_df["declared_monthly_revenue_sum"] = display_df["declared_monthly_revenue_sum"].map(format_brl)
 st.dataframe(display_df, use_container_width=True, hide_index=True)

def page_quality() -> None:
 config = PAGE_CONFIG["Data Quality · Contrôles"]
 page_header(config["title"], config["subtitle"])

 checks = safe_query(
 """
 SELECT 'raw_orders_loaded' AS check_name, COUNT(*)::TEXT AS value, (COUNT(*) > 0) AS passed
 FROM raw.list_orders
 UNION ALL
 SELECT 'dwh_fact_orders_loaded', COUNT(*)::TEXT, (COUNT(*) > 0)
 FROM dwh.fact_orders
 UNION ALL
 SELECT 'fact_order_items_missing_order_key', COUNT(*)::TEXT, (COUNT(*) = 0)
 FROM dwh.fact_order_items WHERE order_key IS NULL
 UNION ALL
 SELECT 'fact_order_items_missing_product_key', COUNT(*)::TEXT, (COUNT(*) = 0)
 FROM dwh.fact_order_items WHERE product_key IS NULL
 UNION ALL
 SELECT 'fact_order_items_missing_seller_key', COUNT(*)::TEXT, (COUNT(*) = 0)
 FROM dwh.fact_order_items WHERE seller_key IS NULL
 UNION ALL
 SELECT 'fact_payments_missing_order_key', COUNT(*)::TEXT, (COUNT(*) = 0)
 FROM dwh.fact_payments WHERE order_key IS NULL
 """
)

```

```

 UNION ALL
 SELECT 'reviews_outside_1_5', COUNT(*)::TEXT, (COUNT(*) = 0)
 FROM dwh.fact_reviews WHERE review_score NOT BETWEEN 1 AND 5
 UNION ALL
 SELECT 'marts_sales_overview_loaded', COUNT(*)::TEXT, (COUNT(*) > 0)
 FROM marts.sales_overview
 UNION ALL
 SELECT 'marts_sales_by_category_loaded', COUNT(*)::TEXT, (COUNT(*) > 0)
 FROM marts.sales_by_category
 """"
)
 if checks.empty:
 show_empty_state("les controles qualite")
 return

 passed = int(checks["passed"].sum())
 total = len(checks)

 section_title("Indicateurs cles")
 metric_grid(
 [
 ("Total checks", format_count(total), None),
 ("Passed", format_count(passed), None),
 ("Failed", format_count(total - passed), None),
]
)

 section_title(
 "Details",
 "Ces controles valident la disponibilite des couches raw, DWH et marts ainsi que la coherence des cles critiques.",
)
 render_quality_table(checks)

def main() -> None:
 page = sidebar()
 if page == "sales":
 page_sales()
 elif page == "payments":
 page_payments()
 elif page == "satisfaction":
 page_satisfaction()
 elif page == "delivery":
 page_delivery()
 elif page == "marketing":
 page_marketing()
 elif page == "quality":
 page_quality()

if __name__ == "__main__":
 main()

```

## dashboard/README.md

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/dashboard/README.md

```

FILE: dashboard/README.md
```markdown
# Streamlit dashboard

This dashboard connects directly to the PostgreSQL Data Warehouse and reads the analytical marts created by the ELT pipeline.

## Run locally

From the repository root:

```bash
pip install -r requirements.txt
python -m src.run_etl --step all
streamlit run dashboard/app.py
```

## Required database objects

The dashboard expects these objects to exist:

```text
marts.sales_overview
marts.sales_by_category
marts.payment_analysis
marts.customer_satisfaction

```

```

marts.delivery_performance
marts.marketing_funnel
```

If a view is missing, run:

```bash
python -m src.run_etl --step marts
```

## Environment

The app reads the same `DATABASE_URL` used by the ETL:

```text
DATABASE_URL=postgresql+psycopg2://olist:olist@localhost:5432/olist_dw
```

```

logs/.gitkeep

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/logs/.gitkeep

```

FILE: logs/.gitkeep
```text
```

```

sql/00_create_schemas_and_tables.sql

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/sql/00_create_schemas_and_tables.sql

```

FILE: sql/00_create_schemas_and_tables.sql
```sql
-- Dedicated schema and warehouse DDL for ETL bootstrap.
-- This file intentionally excludes marts views.
-- It is derived from sql/olist_postgresql_ddl.sql for backward compatibility.

BEGIN;

CREATE SCHEMA IF NOT EXISTS raw;
CREATE SCHEMA IF NOT EXISTS staging;
CREATE SCHEMA IF NOT EXISTS dwh;
CREATE SCHEMA IF NOT EXISTS marts;

-- -----
-- RAW
-- -----

CREATE TABLE IF NOT EXISTS raw.olist_customers (
 customer_id TEXT PRIMARY KEY,
 customer_unique_id TEXT,
 customer_zip_code_prefix TEXT,
 customer_city TEXT,
 customer_state TEXT
);

CREATE TABLE IF NOT EXISTS raw.olist_geolocation (
 geolocation_zip_code_prefix TEXT,
 geolocation_lat NUMERIC(10, 7),
 geolocation_lng NUMERIC(10, 7),
 geolocation_city TEXT,
 geolocation_state TEXT
);

CREATE TABLE IF NOT EXISTS raw.olist_marketing_qualified_leads (
 mql_id TEXT PRIMARY KEY,
 first_contact_date DATE,
 landing_page_id TEXT,
 origin TEXT
);

CREATE TABLE IF NOT EXISTS raw.olist_closed_deals (
 mql_id TEXT PRIMARY KEY,
 seller_id TEXT,
 sdr_id TEXT,
 sr_id TEXT,
 won_date TIMESTAMP,
 business_segment TEXT,
 lead_type TEXT,

```

```

 lead_behaviour_profile TEXT,
 has_company TEXT,
 has_gtin TEXT,
 average_stock TEXT,
 business_type TEXT,
 declared_product_catalog_size NUMERIC(14, 2),
 declared_monthly_revenue NUMERIC(14, 2),
 CHECK (declared_product_catalog_size IS NULL OR declared_product_catalog_size >= 0),
 CHECK (declared_monthly_revenue IS NULL OR declared_monthly_revenue >= 0)
);

CREATE TABLE IF NOT EXISTS raw.olist_orders (
 order_id TEXT PRIMARY KEY,
 customer_id TEXT,
 order_status TEXT,
 order_purchase_timestamp TIMESTAMP,
 order_approved_at TIMESTAMP,
 order_delivered_carrier_date TIMESTAMP,
 order_delivered_customer_date TIMESTAMP,
 order_estimated_delivery_date TIMESTAMP
);

CREATE TABLE IF NOT EXISTS raw.olist_order_items (
 order_id TEXT,
 order_item_id INTEGER,
 product_id TEXT,
 seller_id TEXT,
 shipping_limit_date TIMESTAMP,
 price NUMERIC(12, 2),
 freight_value NUMERIC(12, 2),
 PRIMARY KEY (order_id, order_item_id),
 CHECK (order_item_id > 0),
 CHECK (price >= 0),
 CHECK (freight_value >= 0)
);

CREATE TABLE IF NOT EXISTS raw.olist_order_payments (
 order_id TEXT,
 payment_sequential INTEGER,
 payment_type TEXT,
 payment_installments INTEGER,
 payment_value NUMERIC(12, 2),
 PRIMARY KEY (order_id, payment_sequential),
 CHECK (payment_sequential > 0),
 CHECK (payment_installments >= 0),
 CHECK (payment_value >= 0)
);

CREATE TABLE IF NOT EXISTS raw.olist_order_reviews (
 review_id TEXT,
 order_id TEXT,
 review_score INTEGER,
 review_comment_title TEXT,
 review_comment_message TEXT,
 review_creation_date TIMESTAMP,
 review_answer_timestamp TIMESTAMP,
 CHECK (review_score BETWEEN 1 AND 5)
);

CREATE TABLE IF NOT EXISTS raw.olist_products (
 product_id TEXT PRIMARY KEY,
 product_category_name TEXT,
 product_name_lenght INTEGER,
 product_description_lenght INTEGER,
 product_photos_qty INTEGER,
 product_weight_g NUMERIC(12, 2),
 product_length_cm NUMERIC(12, 2),
 product_height_cm NUMERIC(12, 2),
 product_width_cm NUMERIC(12, 2),
 CHECK (product_name_lenght IS NULL OR product_name_lenght >= 0),
 CHECK (product_description_lenght IS NULL OR product_description_lenght >= 0),
 CHECK (product_photos_qty IS NULL OR product_photos_qty >= 0),
 CHECK (product_weight_g IS NULL OR product_weight_g >= 0),
 CHECK (product_length_cm IS NULL OR product_length_cm >= 0),
 CHECK (product_height_cm IS NULL OR product_height_cm >= 0),
 CHECK (product_width_cm IS NULL OR product_width_cm >= 0)
);

CREATE TABLE IF NOT EXISTS raw.olist_sellers (
 seller_id TEXT PRIMARY KEY,
 seller_zip_code_prefix TEXT,
 seller_city TEXT,
 seller_state TEXT
);

CREATE TABLE IF NOT EXISTS raw.product_category_name_translation (
 product_category_name TEXT PRIMARY KEY,

```

```

 product_category_name_english TEXT
);

CREATE INDEX IF NOT EXISTS idx_raw_geolocation_zip_prefix
 ON raw.olist_geolocation (geolocation_zip_code_prefix);

-- STAGING

CREATE TABLE IF NOT EXISTS staging.stg_customers (
 customer_id TEXT PRIMARY KEY,
 customer_unique_id TEXT NOT NULL,
 zip_prefix TEXT,
 city_name TEXT,
 state_code TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_sellers (
 seller_id TEXT PRIMARY KEY,
 zip_prefix TEXT,
 city_name TEXT,
 state_code TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_geolocation (
 zip_prefix TEXT,
 latitude NUMERIC(10, 7),
 longitude NUMERIC(10, 7),
 city_name TEXT,
 state_code TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_product_categories (
 product_category_name TEXT PRIMARY KEY,
 product_category_name_english TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_products (
 product_id TEXT PRIMARY KEY,
 product_category_name TEXT,
 product_name_length INTEGER,
 product_description_length INTEGER,
 product_photos_qty INTEGER,
 product_weight_g NUMERIC(12, 2),
 product_length_cm NUMERIC(12, 2),
 product_height_cm NUMERIC(12, 2),
 product_width_cm NUMERIC(12, 2),
 CHECK (product_name_length IS NULL OR product_name_length >= 0),
 CHECK (product_description_length IS NULL OR product_description_length >= 0),
 CHECK (product_photos_qty IS NULL OR product_photos_qty >= 0),
 CHECK (product_weight_g IS NULL OR product_weight_g >= 0),
 CHECK (product_length_cm IS NULL OR product_length_cm >= 0),
 CHECK (product_height_cm IS NULL OR product_height_cm >= 0),
 CHECK (product_width_cm IS NULL OR product_width_cm >= 0)
);

CREATE TABLE IF NOT EXISTS staging.stg_orders (
 order_id TEXT PRIMARY KEY,
 customer_id TEXT NOT NULL,
 order_status TEXT,
 order_purchase_ts TIMESTAMP,
 order_approved_ts TIMESTAMP,
 order_delivered_carrier_ts TIMESTAMP,
 order_delivered_customer_ts TIMESTAMP,
 order_estimated_delivery_ts TIMESTAMP
);

CREATE TABLE IF NOT EXISTS staging.stg_order_items (
 order_id TEXT,
 order_item_id INTEGER,
 product_id TEXT,
 seller_id TEXT,
 shipping_limit_ts TIMESTAMP,
 price NUMERIC(12, 2),
 freight_value NUMERIC(12, 2),
 PRIMARY KEY (order_id, order_item_id),
 CHECK (order_item_id > 0),
 CHECK (price >= 0),
 CHECK (freight_value >= 0)
);

CREATE TABLE IF NOT EXISTS staging.stg_payments (
 order_id TEXT,
 payment_sequential INTEGER,
 payment_type TEXT,
 payment_installments INTEGER,

```

```

 payment_value NUMERIC(12, 2),
 PRIMARY KEY (order_id, payment_sequential),
 CHECK (payment_sequential > 0),
 CHECK (payment_installments >= 0),
 CHECK (payment_value >= 0)
);

CREATE TABLE IF NOT EXISTS staging.stg_reviews (
 review_id TEXT,
 order_id TEXT,
 review_score INTEGER,
 review_comment_title TEXT,
 review_comment_message TEXT,
 review_creation_ts TIMESTAMP,
 review_answer_ts TIMESTAMP,
 CHECK (review_score BETWEEN 1 AND 5)
);

CREATE TABLE IF NOT EXISTS staging.stg_marketing_leads (
 mql_id TEXT PRIMARY KEY,
 first_contact_date DATE,
 landing_page_id TEXT,
 origin TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_closed_deals (
 mql_id TEXT PRIMARY KEY,
 seller_id TEXT,
 sdr_id TEXT,
 sr_id TEXT,
 won_ts TIMESTAMP,
 business_segment TEXT,
 lead_type TEXT,
 lead_behaviour_profile TEXT,
 has_company BOOLEAN,
 has_gtin BOOLEAN,
 average_stock TEXT,
 business_type TEXT,
 declared_product_catalog_size NUMERIC(14, 2),
 declared_monthly_revenue NUMERIC(14, 2),
 CHECK (declared_product_catalog_size IS NULL OR declared_product_catalog_size >= 0),
 CHECK (declared_monthly_revenue IS NULL OR declared_monthly_revenue >= 0)
);

-- -----
-- DWH DIMENSIONS
-- -----

CREATE TABLE IF NOT EXISTS dwh.dim_date (
 date_key INTEGER PRIMARY KEY,
 full_date DATE NOT NULL UNIQUE,
 day_of_month SMALLINT NOT NULL,
 day_of_week SMALLINT NOT NULL,
 day_name TEXT NOT NULL,
 week_of_year SMALLINT NOT NULL,
 month_number SMALLINT NOT NULL,
 month_name TEXT NOT NULL,
 quarter_number SMALLINT NOT NULL,
 year_number INTEGER NOT NULL,
 is_weekend BOOLEAN NOT NULL
);

CREATE TABLE IF NOT EXISTS dwh.dim_state (
 state_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 state_code TEXT NOT NULL UNIQUE,
 state_name TEXT
);

CREATE TABLE IF NOT EXISTS dwh.dim_city (
 city_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 city_name TEXT NOT NULL,
 state_key BIGINT NOT NULL REFERENCES dwh.dim_state (state_key),
 UNIQUE (city_name, state_key)
);

CREATE TABLE IF NOT EXISTS dwh.dim_zip_prefix (
 zip_prefix_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 zip_prefix TEXT NOT NULL UNIQUE,
 city_key BIGINT REFERENCES dwh.dim_city (city_key),
 state_key BIGINT REFERENCES dwh.dim_state (state_key),
 latitude NUMERIC(10, 7),
 longitude NUMERIC(10, 7)
);

CREATE TABLE IF NOT EXISTS dwh.dim_customer (
 customer_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 customer_id TEXT NOT NULL UNIQUE,

```

```

 customer_unique_id TEXT NOT NULL,
 zip_prefix_key BIGINT REFERENCES dwh.dim_zip_prefix (zip_prefix_key),
 city_key BIGINT REFERENCES dwh.dim_city (city_key),
 state_key BIGINT REFERENCES dwh.dim_state (state_key)
);

CREATE TABLE IF NOT EXISTS dwh.dim_seller (
 seller_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 seller_id TEXT NOT NULL UNIQUE,
 zip_prefix_key BIGINT REFERENCES dwh.dim_zip_prefix (zip_prefix_key),
 city_key BIGINT REFERENCES dwh.dim_city (city_key),
 state_key BIGINT REFERENCES dwh.dim_state (state_key)
);

CREATE TABLE IF NOT EXISTS dwh.dim_product_category (
 product_category_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 product_category_name TEXT NOT NULL UNIQUE,
 product_category_name_english TEXT
);

CREATE TABLE IF NOT EXISTS dwh.dim_product (
 product_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 product_id TEXT NOT NULL UNIQUE,
 product_category_key BIGINT REFERENCES dwh.dim_product_category (product_category_key),
 product_name_length INTEGER,
 product_description_length INTEGER,
 product_photos_qty INTEGER,
 product_weight_g NUMERIC(12, 2),
 product_length_cm NUMERIC(12, 2),
 product_height_cm NUMERIC(12, 2),
 product_width_cm NUMERIC(12, 2)
);

CREATE TABLE IF NOT EXISTS dwh.dim_order_status (
 order_status_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 order_status TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_payment_type (
 payment_type_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 payment_type TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_marketing_origin (
 marketing_origin_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 origin_name TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_landing_page (
 landing_page_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 landing_page_id TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_business_segment (
 business_segment_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 business_segment TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_lead_type (
 lead_type_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 lead_type TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_lead_profile (
 lead_profile_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 lead_behaviour_profile TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_sales_rep (
 sales_rep_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 sales_rep_id TEXT NOT NULL,
 sales_role TEXT NOT NULL,
 UNIQUE (sales_rep_id, sales_role)
);

-- -----
-- DWH FACTS
-- -----

CREATE TABLE IF NOT EXISTS dwh.fact_marketing_leads (
 marketing_lead_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 mql_id TEXT NOT NULL UNIQUE,
 first_contact_date_key INTEGER REFERENCES dwh.dim_date (date_key),
 landing_page_key BIGINT REFERENCES dwh.dim_landing_page (landing_page_key),
 marketing_origin_key BIGINT REFERENCES dwh.dim_marketing_origin (marketing_origin_key),
 lead_count INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TABLE IF NOT EXISTS dwh.fact_closed_deals (
 closed_deal_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 mql_id TEXT NOT NULL UNIQUE,
 seller_key BIGINT REFERENCES dwh.dim_seller (seller_key),
 won_date_key INTEGER REFERENCES dwh.dim_date (date_key),
 sdr_sales_rep_key BIGINT REFERENCES dwh.dim_sales_rep (sales_rep_key),
 sr_sales_rep_key BIGINT REFERENCES dwh.dim_sales_rep (sales_rep_key),
 business_segment_key BIGINT REFERENCES dwh.dim_business_segment (business_segment_key),
 lead_type_key BIGINT REFERENCES dwh.dim_lead_type (lead_type_key),
 lead_profile_key BIGINT REFERENCES dwh.dim_lead_profile (lead_profile_key),
 has_company BOOLEAN,
 has_gtin BOOLEAN,
 average_stock TEXT,
 business_type TEXT,
 declared_product_catalog_size NUMERIC(14, 2),
 declared_monthly_revenue NUMERIC(14, 2),
 is_active_seller BOOLEAN NOT NULL DEFAULT TRUE,
 deal_count INTEGER NOT NULL DEFAULT 1,
 CHECK (declared_product_catalog_size IS NULL OR declared_product_catalog_size >= 0),
 CHECK (declared_monthly_revenue IS NULL OR declared_monthly_revenue >= 0),
 CHECK (deal_count > 0)
);

```

```

CREATE TABLE IF NOT EXISTS dwh.fact_orders (
 order_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 order_id TEXT NOT NULL UNIQUE,
 customer_key BIGINT REFERENCES dwh.dim_customer (customer_key),
 order_status_key BIGINT REFERENCES dwh.dim_order_status (order_status_key),
 purchase_date_key INTEGER REFERENCES dwh.dim_date (date_key),
 approved_date_key INTEGER REFERENCES dwh.dim_date (date_key),
 delivered_carrier_date_key INTEGER REFERENCES dwh.dim_date (date_key),
 delivered_customer_date_key INTEGER REFERENCES dwh.dim_date (date_key),
 estimated_delivery_date_key INTEGER REFERENCES dwh.dim_date (date_key),
 order_purchase_ts TIMESTAMP,
 order_approved_ts TIMESTAMP,
 order_delivered_carrier_ts TIMESTAMP,
 order_delivered_customer_ts TIMESTAMP,
 order_estimated_delivery_ts TIMESTAMP,
 approval_delay_hours NUMERIC(12, 2),
 days_to_carrier NUMERIC(12, 2),
 days_to_customer NUMERIC(12, 2),
 estimated_delivery_days NUMERIC(12, 2),
 delivery_delay_days NUMERIC(12, 2),
 is_delivered_on_time BOOLEAN,
 is_delivered BOOLEAN NOT NULL DEFAULT FALSE,
 order_count INTEGER NOT NULL DEFAULT 1,
 CHECK (order_count > 0)
);

```

```

CREATE TABLE IF NOT EXISTS dwh.fact_order_items (
 order_item_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 order_id TEXT NOT NULL,
 order_item_id INTEGER NOT NULL,
 order_key BIGINT REFERENCES dwh.fact_orders (order_key),
 product_key BIGINT REFERENCES dwh.dim_product (product_key),
 seller_key BIGINT REFERENCES dwh.dim_seller (seller_key),
 shipping_limit_date_key INTEGER REFERENCES dwh.dim_date (date_key),
 shipping_limit_ts TIMESTAMP,
 item_price NUMERIC(12, 2),
 freight_value NUMERIC(12, 2),
 total_item_value NUMERIC(12, 2),
 item_count INTEGER NOT NULL DEFAULT 1,
 UNIQUE (order_id, order_item_id),
 CHECK (item_price IS NULL OR item_price >= 0),
 CHECK (freight_value IS NULL OR freight_value >= 0),
 CHECK (total_item_value IS NULL OR total_item_value >= 0),
 CHECK (item_count > 0)
);

```

```

CREATE TABLE IF NOT EXISTS dwh.fact_payments (
 payment_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 order_id TEXT NOT NULL,
 payment_sequential INTEGER NOT NULL,
 order_key BIGINT REFERENCES dwh.fact_orders (order_key),
 payment_type_key BIGINT REFERENCES dwh.dim_payment_type (payment_type_key),
 payment_installments INTEGER,
 payment_value NUMERIC(12, 2),
 payment_count INTEGER NOT NULL DEFAULT 1,
 UNIQUE (order_id, payment_sequential),
 CHECK (payment_sequential > 0),
 CHECK (payment_installments IS NULL OR payment_installments >= 0),
 CHECK (payment_value IS NULL OR payment_value >= 0),
 CHECK (payment_count > 0)
);

```

```

CREATE TABLE IF NOT EXISTS dwh.fact_reviews (
 review_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

```

```

review_id TEXT NOT NULL,
order_id TEXT NOT NULL,
order_key BIGINT REFERENCES dwh.fact_orders (order_key),
review_creation_date_key INTEGER REFERENCES dwh.dim_date (date_key),
review_answer_date_key INTEGER REFERENCES dwh.dim_date (date_key),
review_score INTEGER,
review_comment_title TEXT,
review_comment_message TEXT,
review_creation_ts TIMESTAMP,
review_answer_ts TIMESTAMP,
review_response_time_hours NUMERIC(12, 2),
has_review_title BOOLEAN NOT NULL DEFAULT FALSE,
has_review_message BOOLEAN NOT NULL DEFAULT FALSE,
is_positive_review BOOLEAN NOT NULL DEFAULT FALSE,
is_neutral_review BOOLEAN NOT NULL DEFAULT FALSE,
is_negative_review BOOLEAN NOT NULL DEFAULT FALSE,
review_count INTEGER NOT NULL DEFAULT 1,
CHECK (review_score BETWEEN 1 AND 5),
CHECK (review_count > 0)
);

CREATE INDEX IF NOT EXISTS idx_fact_orders_customer_key
ON dwh.fact_orders (customer_key);

CREATE INDEX IF NOT EXISTS idx_fact_order_items_order_key
ON dwh.fact_order_items (order_key);

CREATE INDEX IF NOT EXISTS idx_fact_order_items_product_key
ON dwh.fact_order_items (product_key);

CREATE INDEX IF NOT EXISTS idx_fact_payments_order_key
ON dwh.fact_payments (order_key);

CREATE INDEX IF NOT EXISTS idx_fact_reviews_order_key
ON dwh.fact_reviews (order_key);

CREATE INDEX IF NOT EXISTS idx_fact_closed_deals_seller_key
ON dwh.fact_closed_deals (seller_key);

COMMIT;

```

## sql/01\_create\_marts.sql

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/sql/01\_create\_marts.sql

```

FILE: sql/01_create_marts.sql
```sql
-- Dedicated analytical marts for the Streamlit dashboard.
-- This file can be executed with: python -m src.run_etl --step marts

CREATE SCHEMA IF NOT EXISTS marts;

CREATE OR REPLACE VIEW marts.sales_overview AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    os.order_status,
    COUNT(DISTINCT fo.order_id) AS order_count,
    COUNT(foi.order_item_key) AS item_count,
    COALESCE(SUM(foi.item_price), 0) AS gross_item_value,
    COALESCE(SUM(foi.freight_value), 0) AS gross_freight_value,
    COALESCE(SUM(foi.item_price + foi.freight_value), 0) AS gross_revenue
FROM dwh.fact_orders fo
LEFT JOIN dwh.fact_order_items foi ON fo.order_key = foi.order_key
LEFT JOIN dwh.dim_date dd ON fo.purchase_date_key = dd.date_key
LEFT JOIN dwh.dim_order_status os ON fo.order_status_key = os.order_status_key
GROUP BY dd.year_number, dd.month_number, dd.month_name, os.order_status;

CREATE OR REPLACE VIEW marts.sales_by_category AS
SELECT
    COALESCE(NULLIF(dpc.product_category_name_english, ''), dpc.product_category_name, 'unknown') AS product_category,
    COUNT(foi.order_item_key) AS item_count,
    COUNT(DISTINCT foi.order_id) AS order_count,
    COALESCE(SUM(foi.item_price), 0) AS gross_item_value,
    COALESCE(SUM(foi.freight_value), 0) AS gross_freight_value,
    COALESCE(SUM(foi.item_price + foi.freight_value), 0) AS gross_revenue
FROM dwh.fact_order_items foi
LEFT JOIN dwh.dim_product dp ON foi.product_key = dp.product_key
LEFT JOIN dwh.dim_product_category dpc ON dp.product_category_key = dpc.product_category_key
GROUP BY COALESCE(NULLIF(dpc.product_category_name_english, ''), dpc.product_category_name, 'unknown');
```

```

CREATE OR REPLACE VIEW marts.payment_analysis AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    pt.payment_type,
    fp.payment_installments,
    COUNT(*) AS payment_count,
    COALESCE(SUM(fp.payment_value), 0) AS total_payment_value,
    AVG(fp.payment_value) AS avg_payment_value
FROM dwh.fact_payments fp
LEFT JOIN dwh.fact_orders fo ON fp.order_key = fo.order_key
LEFT JOIN dwh.dim_date dd ON fo.purchase_date_key = dd.date_key
LEFT JOIN dwh.dim_payment_type pt ON fp.payment_type_key = pt.payment_type_key
GROUP BY dd.year_number, dd.month_number, dd.month_name, pt.payment_type, fp.payment_installments;

CREATE OR REPLACE VIEW marts.customer_satisfaction AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    fr.review_score,
    COUNT(*) AS review_count,
    AVG(fr.review_score::NUMERIC) AS avg_review_score
FROM dwh.fact_reviews fr
LEFT JOIN dwh.dim_date dd ON fr.review_creation_date_key = dd.date_key
GROUP BY dd.year_number, dd.month_number, dd.month_name, fr.review_score;

CREATE OR REPLACE VIEW marts.delivery_performance AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    os.order_status,
    COUNT(*) AS delivered_orders,
    AVG(EXTRACT(EPOCH FROM (fo.order_delivered_customer_ts - fo.order_purchase_ts)) / 86400.0) AS avg_days_purchase_to_delivered,
    AVG(EXTRACT(EPOCH FROM (fo.order_delivered_customer_ts - fo.order_estimated_delivery_ts)) / 86400.0) AS avg_days_vs_estimated,
    AVG(CASE WHEN fo.is_delivered_on_time THEN 1.0 ELSE 0.0 END) AS on_time_rate
FROM dwh.fact_orders fo
LEFT JOIN dwh.dim_date dd ON fo.purchase_date_key = dd.date_key
LEFT JOIN dwh.dim_order_status os ON fo.order_status_key = os.order_status_key
WHERE fo.order_delivered_customer_ts IS NOT NULL
GROUP BY dd.year_number, dd.month_number, dd.month_name, os.order_status;

CREATE OR REPLACE VIEW marts.marketing_funnel AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    mo.origin_name AS marketing_origin,
    COUNT(DISTINCT fml.mql_id) AS mql_count,
    COUNT(DISTINCT fcd.mql_id) AS won_deal_count,
    COALESCE(SUM(fcd.declared_monthly_revenue), 0) AS declared_monthly_revenue_sum,
    CASE
        WHEN COUNT(DISTINCT fml.mql_id) = 0 THEN 0
        ELSE COUNT(DISTINCT fcd.mql_id)::NUMERIC / COUNT(DISTINCT fml.mql_id)
    END AS conversion_rate
FROM dwh.fact_marketing_leads fml
LEFT JOIN dwh.fact_closed_deals fcd ON fml.mql_id = fcd.mql_id
LEFT JOIN dwh.dim_date dd ON fml.first_contact_date_key = dd.date_key
LEFT JOIN dwh.dim_marketing_origin mo ON fml.marketing_origin_key = mo.marketing_origin_key
GROUP BY dd.year_number, dd.month_number, dd.month_name, mo.origin_name;
...

```

sql/99_quality_checks.sql

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/sql/99_quality_checks.sql

```

FILE: sql/99_quality_checks.sql
```sql
-- Manual quality checks for the Olist Data Warehouse.
-- Run in psql or any SQL client after the ELT load.

SELECT 'raw_orders_loaded' AS check_name, COUNT(*)::TEXT AS value, (COUNT(*) > 0) AS passed
FROM raw.olist_orders
UNION ALL
SELECT 'dwh_fact_orders_loaded', COUNT(*)::TEXT, (COUNT(*) > 0)
FROM dwh.fact_orders
UNION ALL
SELECT 'fact_order_items_missing_order_key', COUNT(*)::TEXT, (COUNT(*) = 0)
FROM dwh.fact_order_items WHERE order_key IS NULL
UNION ALL
SELECT 'fact_order_items_missing_product_key', COUNT(*)::TEXT, (COUNT(*) = 0)

```

```

FROM dwh.fact_order_items WHERE product_key IS NULL
UNION ALL
SELECT 'fact_order_items_missing_seller_key', COUNT(*)::TEXT, (COUNT(*) = 0)
FROM dwh.fact_order_items WHERE seller_key IS NULL
UNION ALL
SELECT 'fact_payments_missing_order_key', COUNT(*)::TEXT, (COUNT(*) = 0)
FROM dwh.fact_payments WHERE order_key IS NULL
UNION ALL
SELECT 'fact_reviews_missing_order_key', COUNT(*)::TEXT, (COUNT(*) = 0)
FROM dwh.fact_reviews WHERE order_key IS NULL
UNION ALL
SELECT 'reviews_outside_1_5', COUNT(*)::TEXT, (COUNT(*) = 0)
FROM dwh.fact_reviews WHERE review_score NOT BETWEEN 1 AND 5
UNION ALL
SELECT 'marts_sales_overview_loaded', COUNT(*)::TEXT, (COUNT(*) > 0)
FROM marts.sales_overview
UNION ALL
SELECT 'marts_payment_analysis_loaded', COUNT(*)::TEXT, (COUNT(*) > 0)
FROM marts.payment_analysis
UNION ALL
SELECT 'marts_customer_satisfaction_loaded', COUNT(*)::TEXT, (COUNT(*) > 0)
FROM marts.customer_satisfaction;
...

```

## sql/olist\_postgresql\_ddl.sql

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/sql/olist\_postgresql\_ddl.sql

```

FILE: sql/olist_postgresql_ddl.sql
```sql
-- Olist PostgreSQL DDL
-- Legacy/backward-compatible combined DDL.
-- Prefer sql/00_create_schemas_and_tables.sql for schema bootstrap and
-- sql/01_create_marts.sql for marts creation in current ETL flows.
-- Target schemas:
--   raw, staging, dwh, marts
--
-- This script defines:
--   1. Raw landing tables that mirror the CSV files
--   2. Staging tables with cleaned and typed structures
--   3. Core warehouse dimensions and facts
--   4. Dashboard marts as views

BEGIN;

CREATE SCHEMA IF NOT EXISTS raw;
CREATE SCHEMA IF NOT EXISTS staging;
CREATE SCHEMA IF NOT EXISTS dwh;
CREATE SCHEMA IF NOT EXISTS marts;

-- -----
-- RAW
-- -----

CREATE TABLE IF NOT EXISTS raw.olist_customers (
    customer_id TEXT PRIMARY KEY,
    customer_unique_id TEXT,
    customer_zip_code_prefix TEXT,
    customer_city TEXT,
    customer_state TEXT
);

CREATE TABLE IF NOT EXISTS raw.olist_geolocation (
    geolocation_zip_code_prefix TEXT,
    geolocation_lat NUMERIC(10, 7),
    geolocation_lng NUMERIC(10, 7),
    geolocation_city TEXT,
    geolocation_state TEXT
);

CREATE TABLE IF NOT EXISTS raw.olist_marketing_qualified_leads (
    mql_id TEXT PRIMARY KEY,
    first_contact_date DATE,
    landing_page_id TEXT,
    origin TEXT
);

CREATE TABLE IF NOT EXISTS raw.olist_closed_deals (
    mql_id TEXT PRIMARY KEY,
    seller_id TEXT,
    sdr_id TEXT,
    sr_id TEXT,
    won_date TIMESTAMP,

```

```

        business_segment TEXT,
        lead_type TEXT,
        lead_behaviour_profile TEXT,
        has_company TEXT,
        has_gtin TEXT,
        average_stock TEXT,
        business_type TEXT,
        declared_product_catalog_size NUMERIC(14, 2),
        declared_monthly_revenue NUMERIC(14, 2),
        CHECK (declared_product_catalog_size IS NULL OR declared_product_catalog_size >= 0),
        CHECK (declared_monthly_revenue IS NULL OR declared_monthly_revenue >= 0)
    );

CREATE TABLE IF NOT EXISTS raw.olist_orders (
    order_id TEXT PRIMARY KEY,
    customer_id TEXT,
    order_status TEXT,
    order_purchase_timestamp TIMESTAMP,
    order_approved_at TIMESTAMP,
    order_delivered_carrier_date TIMESTAMP,
    order_delivered_customer_date TIMESTAMP,
    order_estimated_delivery_date TIMESTAMP
);

CREATE TABLE IF NOT EXISTS raw.olist_order_items (
    order_id TEXT,
    order_item_id INTEGER,
    product_id TEXT,
    seller_id TEXT,
    shipping_limit_date TIMESTAMP,
    price NUMERIC(12, 2),
    freight_value NUMERIC(12, 2),
    PRIMARY KEY (order_id, order_item_id),
    CHECK (order_item_id > 0),
    CHECK (price >= 0),
    CHECK (freight_value >= 0)
);

CREATE TABLE IF NOT EXISTS raw.olist_order_payments (
    order_id TEXT,
    payment_sequential INTEGER,
    payment_type TEXT,
    payment_installments INTEGER,
    payment_value NUMERIC(12, 2),
    PRIMARY KEY (order_id, payment_sequential),
    CHECK (payment_sequential > 0),
    CHECK (payment_installments >= 0),
    CHECK (payment_value >= 0)
);

CREATE TABLE IF NOT EXISTS raw.olist_order_reviews (
    review_id TEXT,
    order_id TEXT,
    review_score INTEGER,
    review_comment_title TEXT,
    review_comment_message TEXT,
    review_creation_date TIMESTAMP,
    review_answer_timestamp TIMESTAMP,
    CHECK (review_score BETWEEN 1 AND 5)
);

CREATE TABLE IF NOT EXISTS raw.olist_products (
    product_id TEXT PRIMARY KEY,
    product_category_name TEXT,
    product_name_lenght INTEGER,
    product_description_lenght INTEGER,
    product_photos_qty INTEGER,
    product_weight_g NUMERIC(12, 2),
    product_length_cm NUMERIC(12, 2),
    product_height_cm NUMERIC(12, 2),
    product_width_cm NUMERIC(12, 2),
    CHECK (product_name_lenght IS NULL OR product_name_lenght >= 0),
    CHECK (product_description_lenght IS NULL OR product_description_lenght >= 0),
    CHECK (product_photos_qty IS NULL OR product_photos_qty >= 0),
    CHECK (product_weight_g IS NULL OR product_weight_g >= 0),
    CHECK (product_length_cm IS NULL OR product_length_cm >= 0),
    CHECK (product_height_cm IS NULL OR product_height_cm >= 0),
    CHECK (product_width_cm IS NULL OR product_width_cm >= 0)
);

CREATE TABLE IF NOT EXISTS raw.olist_sellers (
    seller_id TEXT PRIMARY KEY,
    seller_zip_code_prefix TEXT,
    seller_city TEXT,
    seller_state TEXT
);

```

```

CREATE TABLE IF NOT EXISTS raw.product_category_name_translation (
    product_category_name TEXT PRIMARY KEY,
    product_category_name_english TEXT
);

CREATE INDEX IF NOT EXISTS idx_raw_geolocation_zip_prefix
    ON raw.olist_geolocation (geolocation_zip_code_prefix);

-----
-- STAGING
-----

CREATE TABLE IF NOT EXISTS staging.stg_customers (
    customer_id TEXT PRIMARY KEY,
    customer_unique_id TEXT NOT NULL,
    zip_prefix TEXT,
    city_name TEXT,
    state_code TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_sellers (
    seller_id TEXT PRIMARY KEY,
    zip_prefix TEXT,
    city_name TEXT,
    state_code TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_geolocation (
    zip_prefix TEXT,
    latitude NUMERIC(10, 7),
    longitude NUMERIC(10, 7),
    city_name TEXT,
    state_code TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_product_categories (
    product_category_name TEXT PRIMARY KEY,
    product_category_name_english TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_products (
    product_id TEXT PRIMARY KEY,
    product_category_name TEXT,
    product_name_length INTEGER,
    product_description_length INTEGER,
    product_photos_qty INTEGER,
    product_weight_g NUMERIC(12, 2),
    product_length_cm NUMERIC(12, 2),
    product_height_cm NUMERIC(12, 2),
    product_width_cm NUMERIC(12, 2),
    CHECK (product_name_length IS NULL OR product_name_length >= 0),
    CHECK (product_description_length IS NULL OR product_description_length >= 0),
    CHECK (product_photos_qty IS NULL OR product_photos_qty >= 0),
    CHECK (product_weight_g IS NULL OR product_weight_g >= 0),
    CHECK (product_length_cm IS NULL OR product_length_cm >= 0),
    CHECK (product_height_cm IS NULL OR product_height_cm >= 0),
    CHECK (product_width_cm IS NULL OR product_width_cm >= 0)
);

CREATE TABLE IF NOT EXISTS staging.stg_orders (
    order_id TEXT PRIMARY KEY,
    customer_id TEXT NOT NULL,
    order_status TEXT,
    order_purchase_ts TIMESTAMP,
    order_approved_ts TIMESTAMP,
    order_delivered_carrier_ts TIMESTAMP,
    order_delivered_customer_ts TIMESTAMP,
    order_estimated_delivery_ts TIMESTAMP
);

CREATE TABLE IF NOT EXISTS staging.stg_order_items (
    order_id TEXT,
    order_item_id INTEGER,
    product_id TEXT,
    seller_id TEXT,
    shipping_limit_ts TIMESTAMP,
    price NUMERIC(12, 2),
    freight_value NUMERIC(12, 2),
    PRIMARY KEY (order_id, order_item_id),
    CHECK (order_item_id > 0),
    CHECK (price >= 0),
    CHECK (freight_value >= 0)
);

CREATE TABLE IF NOT EXISTS staging.stg_payments (
    order_id TEXT,
    payment_sequential INTEGER,

```

```

        payment_type TEXT,
        payment_installments INTEGER,
        payment_value NUMERIC(12, 2),
        PRIMARY KEY (order_id, payment_sequential),
        CHECK (payment_sequential > 0),
        CHECK (payment_installments >= 0),
        CHECK (payment_value >= 0)
    );

CREATE TABLE IF NOT EXISTS staging.stg_reviews (
    review_id TEXT,
    order_id TEXT,
    review_score INTEGER,
    review_comment_title TEXT,
    review_comment_message TEXT,
    review_creation_ts TIMESTAMP,
    review_answer_ts TIMESTAMP,
    CHECK (review_score BETWEEN 1 AND 5)
);

CREATE TABLE IF NOT EXISTS staging.stg_marketing_leads (
    mql_id TEXT PRIMARY KEY,
    first_contact_date DATE,
    landing_page_id TEXT,
    origin TEXT
);

CREATE TABLE IF NOT EXISTS staging.stg_closed_deals (
    mql_id TEXT PRIMARY KEY,
    seller_id TEXT,
    sdr_id TEXT,
    sr_id TEXT,
    won_ts TIMESTAMP,
    business_segment TEXT,
    lead_type TEXT,
    lead_behaviour_profile TEXT,
    has_company BOOLEAN,
    has_gtin BOOLEAN,
    average_stock TEXT,
    business_type TEXT,
    declared_product_catalog_size NUMERIC(14, 2),
    declared_monthly_revenue NUMERIC(14, 2),
    CHECK (declared_product_catalog_size IS NULL OR declared_product_catalog_size >= 0),
    CHECK (declared_monthly_revenue IS NULL OR declared_monthly_revenue >= 0)
);

-- -----
-- DWH DIMENSIONS
-- -----

CREATE TABLE IF NOT EXISTS dwh.dim_date (
    date_key INTEGER PRIMARY KEY,
    full_date DATE NOT NULL UNIQUE,
    day_of_month SMALLINT NOT NULL,
    day_of_week SMALLINT NOT NULL,
    day_name TEXT NOT NULL,
    week_of_year SMALLINT NOT NULL,
    month_number SMALLINT NOT NULL,
    month_name TEXT NOT NULL,
    quarter_number SMALLINT NOT NULL,
    year_number INTEGER NOT NULL,
    is_weekend BOOLEAN NOT NULL
);

CREATE TABLE IF NOT EXISTS dwh.dim_state (
    state_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    state_code TEXT NOT NULL UNIQUE,
    state_name TEXT
);

CREATE TABLE IF NOT EXISTS dwh.dim_city (
    city_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    city_name TEXT NOT NULL,
    state_key BIGINT NOT NULL REFERENCES dwh.dim_state (state_key),
    UNIQUE (city_name, state_key)
);

CREATE TABLE IF NOT EXISTS dwh.dim_zip_prefix (
    zip_prefix_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    zip_prefix TEXT NOT NULL UNIQUE,
    city_key BIGINT REFERENCES dwh.dim_city (city_key),
    state_key BIGINT REFERENCES dwh.dim_state (state_key),
    latitude NUMERIC(10, 7),
    longitude NUMERIC(10, 7)
);

CREATE TABLE IF NOT EXISTS dwh.dim_customer (

```

```

        customer_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        customer_id TEXT NOT NULL UNIQUE,
        customer_unique_id TEXT NOT NULL,
        zip_prefix_key BIGINT REFERENCES dwh.dim_zip_prefix (zip_prefix_key),
        city_key BIGINT REFERENCES dwh.dim_city (city_key),
        state_key BIGINT REFERENCES dwh.dim_state (state_key)
    );

CREATE TABLE IF NOT EXISTS dwh.dim_seller (
    seller_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    seller_id TEXT NOT NULL UNIQUE,
    zip_prefix_key BIGINT REFERENCES dwh.dim_zip_prefix (zip_prefix_key),
    city_key BIGINT REFERENCES dwh.dim_city (city_key),
    state_key BIGINT REFERENCES dwh.dim_state (state_key)
);

CREATE TABLE IF NOT EXISTS dwh.dim_product_category (
    product_category_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_category_name TEXT NOT NULL UNIQUE,
    product_category_name_english TEXT
);

CREATE TABLE IF NOT EXISTS dwh.dim_product (
    product_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id TEXT NOT NULL UNIQUE,
    product_category_key BIGINT REFERENCES dwh.dim_product_category (product_category_key),
    product_name_length INTEGER,
    product_description_length INTEGER,
    product_photos_qty INTEGER,
    product_weight_g NUMERIC(12, 2),
    product_length_cm NUMERIC(12, 2),
    product_height_cm NUMERIC(12, 2),
    product_width_cm NUMERIC(12, 2)
);

CREATE TABLE IF NOT EXISTS dwh.dim_order_status (
    order_status_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_status TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_payment_type (
    payment_type_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    payment_type TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_marketing_origin (
    marketing_origin_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    origin_name TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_landing_page (
    landing_page_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    landing_page_id TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_business_segment (
    business_segment_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    business_segment TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_lead_type (
    lead_type_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    lead_type TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_lead_profile (
    lead_profile_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    lead_behaviour_profile TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS dwh.dim_sales_rep (
    sales_rep_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sales_rep_id TEXT NOT NULL,
    sales_role TEXT NOT NULL,
    UNIQUE (sales_rep_id, sales_role)
);

-- -----
-- DWH FACTS
-- -----

CREATE TABLE IF NOT EXISTS dwh.fact_marketing_leads (
    marketing_lead_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    mql_id TEXT NOT NULL UNIQUE,
    first_contact_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    landing_page_key BIGINT REFERENCES dwh.dim_landing_page (landing_page_key),
    marketing_origin_key BIGINT REFERENCES dwh.dim_marketing_origin (marketing_origin_key),

```

```

        lead_count INTEGER NOT NULL DEFAULT 1
    );

CREATE TABLE IF NOT EXISTS dwh.fact_closed_deals (
    closed_deal_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    mql_id TEXT NOT NULL UNIQUE,
    seller_key BIGINT REFERENCES dwh.dim_seller (seller_key),
    won_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    sdr_sales_rep_key BIGINT REFERENCES dwh.dim_sales_rep (sales_rep_key),
    sr_sales_rep_key BIGINT REFERENCES dwh.dim_sales_rep (sales_rep_key),
    business_segment_key BIGINT REFERENCES dwh.dim_business_segment (business_segment_key),
    lead_type_key BIGINT REFERENCES dwh.dim_lead_type (lead_type_key),
    lead_profile_key BIGINT REFERENCES dwh.dim_lead_profile (lead_profile_key),
    has_company BOOLEAN,
    has_gtin BOOLEAN,
    average_stock TEXT,
    business_type TEXT,
    declared_product_catalog_size NUMERIC(14, 2),
    declared_monthly_revenue NUMERIC(14, 2),
    is_active_seller BOOLEAN NOT NULL DEFAULT TRUE,
    deal_count INTEGER NOT NULL DEFAULT 1,
    CHECK (declared_product_catalog_size IS NULL OR declared_product_catalog_size >= 0),
    CHECK (declared_monthly_revenue IS NULL OR declared_monthly_revenue >= 0),
    CHECK (deal_count > 0)
);

CREATE TABLE IF NOT EXISTS dwh.fact_orders (
    order_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id TEXT NOT NULL UNIQUE,
    customer_key BIGINT REFERENCES dwh.dim_customer (customer_key),
    order_status_key BIGINT REFERENCES dwh.dim_order_status (order_status_key),
    purchase_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    approved_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    delivered_carrier_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    delivered_customer_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    estimated_delivery_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    order_purchase_ts TIMESTAMP,
    order_approved_ts TIMESTAMP,
    order_delivered_carrier_ts TIMESTAMP,
    order_delivered_customer_ts TIMESTAMP,
    order_estimated_delivery_ts TIMESTAMP,
    approval_delay_hours NUMERIC(12, 2),
    days_to_carrier NUMERIC(12, 2),
    days_to_customer NUMERIC(12, 2),
    estimated_delivery_days NUMERIC(12, 2),
    delivery_delay_days NUMERIC(12, 2),
    is_delivered_on_time BOOLEAN,
    is_delivered BOOLEAN NOT NULL DEFAULT FALSE,
    order_count INTEGER NOT NULL DEFAULT 1,
    CHECK (order_count > 0)
);

CREATE TABLE IF NOT EXISTS dwh.fact_order_items (
    order_item_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id TEXT NOT NULL,
    order_item_id INTEGER NOT NULL,
    order_key BIGINT REFERENCES dwh.fact_orders (order_key),
    product_key BIGINT REFERENCES dwh.dim_product (product_key),
    seller_key BIGINT REFERENCES dwh.dim_seller (seller_key),
    shipping_limit_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    shipping_limit_ts TIMESTAMP,
    item_price NUMERIC(12, 2),
    freight_value NUMERIC(12, 2),
    total_item_value NUMERIC(12, 2),
    item_count INTEGER NOT NULL DEFAULT 1,
    UNIQUE (order_id, order_item_id),
    CHECK (item_price IS NULL OR item_price >= 0),
    CHECK (freight_value IS NULL OR freight_value >= 0),
    CHECK (total_item_value IS NULL OR total_item_value >= 0),
    CHECK (item_count > 0)
);

CREATE TABLE IF NOT EXISTS dwh.fact_payments (
    payment_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id TEXT NOT NULL,
    payment_sequential INTEGER NOT NULL,
    order_key BIGINT REFERENCES dwh.fact_orders (order_key),
    payment_type_key BIGINT REFERENCES dwh.dim_payment_type (payment_type_key),
    payment_installments INTEGER,
    payment_value NUMERIC(12, 2),
    payment_count INTEGER NOT NULL DEFAULT 1,
    UNIQUE (order_id, payment_sequential),
    CHECK (payment_sequential > 0),
    CHECK (payment_installments IS NULL OR payment_installments >= 0),
    CHECK (payment_value IS NULL OR payment_value >= 0),
    CHECK (payment_count > 0)
);

```

```

CREATE TABLE IF NOT EXISTS dwh.fact_reviews (
    review_key BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    review_id TEXT NOT NULL,
    order_id TEXT NOT NULL,
    order_key BIGINT REFERENCES dwh.fact_orders (order_key),
    review_creation_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    review_answer_date_key INTEGER REFERENCES dwh.dim_date (date_key),
    review_score INTEGER,
    review_comment_title TEXT,
    review_comment_message TEXT,
    review_creation_ts TIMESTAMP,
    review_answer_ts TIMESTAMP,
    review_response_time_hours NUMERIC(12, 2),
    has_review_title BOOLEAN NOT NULL DEFAULT FALSE,
    has_review_message BOOLEAN NOT NULL DEFAULT FALSE,
    is_positive_review BOOLEAN NOT NULL DEFAULT FALSE,
    is_neutral_review BOOLEAN NOT NULL DEFAULT FALSE,
    is_negative_review BOOLEAN NOT NULL DEFAULT FALSE,
    review_count INTEGER NOT NULL DEFAULT 1,
    CHECK (review_score BETWEEN 1 AND 5),
    CHECK (review_count > 0)
);

CREATE INDEX IF NOT EXISTS idx_fact_orders_customer_key
ON dwh.fact_orders (customer_key);

CREATE INDEX IF NOT EXISTS idx_fact_order_items_order_key
ON dwh.fact_order_items (order_key);

CREATE INDEX IF NOT EXISTS idx_fact_order_items_product_key
ON dwh.fact_order_items (product_key);

CREATE INDEX IF NOT EXISTS idx_fact_payments_order_key
ON dwh.fact_payments (order_key);

CREATE INDEX IF NOT EXISTS idx_fact_reviews_order_key
ON dwh.fact_reviews (order_key);

CREATE INDEX IF NOT EXISTS idx_fact_closed_deals_seller_key
ON dwh.fact_closed_deals (seller_key);

-----
-- MARTS
-----

CREATE OR REPLACE VIEW marts.sales_overview AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    os.order_status,
    COUNT(DISTINCT fo.order_id) AS order_count,
    COUNT(foi.order_item_key) AS item_count,
    SUM(foi.item_price) AS gross_item_value,
    SUM(foi.freight_value) AS gross_freight_value,
    SUM(foi.item_price + foi.freight_value) AS gross_revenue
FROM dwh.fact_orders fo
LEFT JOIN dwh.fact_order_items foi
    ON fo.order_key = foi.order_key
LEFT JOIN dwh.dim_date dd
    ON fo.purchase_date_key = dd.date_key
LEFT JOIN dwh.dim_order_status os
    ON fo.order_status_key = os.order_status_key
GROUP BY
    dd.year_number,
    dd.month_number,
    dd.month_name,
    os.order_status;

CREATE OR REPLACE VIEW marts.delivery_performance AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    os.order_status,
    COUNT(*) AS delivered_orders,
    AVG(
        EXTRACT(EPOCH FROM (
            fo.order_delivered_customer_ts - fo.order_purchase_ts
        )) / 86400.0
    ) AS avg_days_purchase_to_delivery,
    AVG(
        EXTRACT(EPOCH FROM (
            fo.order_delivered_customer_ts - fo.order_estimated_delivery_ts
        )) / 86400.0
    ) AS avg_days_vs_estimate
FROM dwh.fact_orders fo

```

```

LEFT JOIN dwh.dim_date dd
    ON fo.purchase_date_key = dd.date_key
LEFT JOIN dwh.dim_order_status os
    ON fo.order_status_key = os.order_status_key
WHERE fo.order_delivered_customer_ts IS NOT NULL
GROUP BY
    dd.year_number,
    dd.month_number,
    dd.month_name,
    os.order_status;

CREATE OR REPLACE VIEW marts.customer_satisfaction AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    fr.review_score,
    COUNT(*) AS review_count,
    AVG(fr.review_score::NUMERIC) AS avg_review_score
FROM dwh.fact_reviews fr
LEFT JOIN dwh.dim_date dd
    ON fr.review_creation_date_key = dd.date_key
GROUP BY
    dd.year_number,
    dd.month_number,
    dd.month_name,
    fr.review_score;

CREATE OR REPLACE VIEW marts.payment_analysis AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    pt.payment_type,
    fp.payment_installments,
    COUNT(*) AS payment_count,
    SUM(fp.payment_value) AS total_payment_value,
    AVG(fp.payment_value) AS avg_payment_value
FROM dwh.fact_payments fp
LEFT JOIN dwh.fact_orders fo
    ON fp.order_key = fo.order_key
LEFT JOIN dwh.dim_date dd
    ON fo.purchase_date_key = dd.date_key
LEFT JOIN dwh.dim_payment_type pt
    ON fp.payment_type_key = pt.payment_type_key
GROUP BY
    dd.year_number,
    dd.month_number,
    dd.month_name,
    pt.payment_type,
    fp.payment_installments;

CREATE OR REPLACE VIEW marts.marketing_funnel AS
SELECT
    dd.year_number,
    dd.month_number,
    dd.month_name,
    mo.origin_name AS marketing_origin,
    COUNT(DISTINCT fml.mql_id) AS mql_count,
    COUNT(DISTINCT fcd.mql_id) AS won_deal_count,
    SUM(fcd.declared_monthly_revenue) AS declared_monthly_revenue_sum
FROM dwh.fact_marketing_leads fml
LEFT JOIN dwh.fact_closed_deals fcd
    ON fml.mql_id = fcd.mql_id
LEFT JOIN dwh.dim_date dd
    ON fml.first_contact_date_key = dd.date_key
LEFT JOIN dwh.dim_marketing_origin mo
    ON fml.marketing_origin_key = mo.marketing_origin_key
GROUP BY
    dd.year_number,
    dd.month_number,
    dd.month_name,
    mo.origin_name;

COMMIT;

...

```

src/___init___py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/src/___init___py

```

FILE: src/___init___py
```python

```

```
...
```

## src/config.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/src/config.py

```
FILE: src/config.py
```python
from __future__ import annotations

import os
from dataclasses import dataclass
from pathlib import Path

try:
    from dotenv import load_dotenv
except ModuleNotFoundError: # pragma: no cover - optional dependency
    def load_dotenv() -> bool:
        return False

load_dotenv()

@dataclass(frozen=True)
class Settings:
    database_url: str = os.getenv(
        "DATABASE_URL",
        "postgresql+psycpg2://olist:olist@localhost:5432/olist_dw",
    )
    raw_data_dir: Path = Path(os.getenv("RAW_DATA_DIR", "data/raw"))
    project_root: Path = Path(__file__).resolve().parents[1]

    @property
    def sql_dir(self) -> Path:
        return self.project_root / "sql"

    @property
    def schema_sql_path(self) -> Path:
        candidates = [
            self.sql_dir / "00_create_schemas_and_tables.sql",
            self.sql_dir / "olist_postgresql_ddl.sql",
        ]
        for path in candidates:
            if path.exists():
                return path
        return candidates[0]

    @property
    def marts_sql_path(self) -> Path:
        candidates = [
            self.sql_dir / "01_create_marts.sql",
            self.sql_dir / "olist_postgresql_ddl.sql",
        ]
        for path in candidates:
            if path.exists():
                return path
        return candidates[0]

    @property
    def quality_sql_path(self) -> Path:
        return self.sql_dir / "99_quality_checks.sql"

settings = Settings()
...

```

src/db.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/src/db.py

```
FILE: src/db.py
```python
from __future__ import annotations

from pathlib import Path
from typing import Iterable

from sqlalchemy import create_engine, text
from sqlalchemy.engine import Engine

```

```

from .config import settings

def get_engine() -> Engine:
 return create_engine(settings.database_url, pool_pre_ping=True, future=True)

def execute_sql_file(engine: Engine, path: Path) -> None:
 sql = path.read_text(encoding="utf-8")
 statements = [stmt.strip() for stmt in sql.split(";") if stmt.strip()]
 with engine.begin() as conn:
 for stmt in statements:
 conn.execute(text(stmt))

def execute_many(engine: Engine, statements: Iterable[str]) -> None:
 with engine.begin() as conn:
 for stmt in statements:
 conn.execute(text(stmt))

def fetch_scalar(engine: Engine, sql: str):
 with engine.connect() as conn:
 return conn.execute(text(sql)).scalar()

...

```

## src/extract\_load\_raw.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/src/extract\_load\_raw.py

```

FILE: src/extract_load_raw.py
```python
from __future__ import annotations

import logging
from pathlib import Path

from sqlalchemy import text
from sqlalchemy.engine import Engine

from .config import settings

logger = logging.getLogger(__name__)

RAW_TABLES = {
    "olist_customers_dataset.csv": (
        "raw.olist_customers",
        [
            "customer_id",
            "customer_unique_id",
            "customer_zip_code_prefix",
            "customer_city",
            "customer_state",
        ],
    ),
    "olist_geolocation_dataset.csv": (
        "raw.olist_geolocation",
        [
            "geolocation_zip_code_prefix",
            "geolocation_lat",
            "geolocation_lng",
            "geolocation_city",
            "geolocation_state",
        ],
    ),
    "olist_marketing_qualified_leads_dataset.csv": (
        "raw.olist_marketing_qualified_leads",
        ["mql_id", "first_contact_date", "landing_page_id", "origin"],
    ),
    "olist_closed_deals_dataset.csv": (
        "raw.olist_closed_deals",
        [
            "mql_id",
            "seller_id",
            "sdr_id",
            "sr_id",
            "won_date",
            "business_segment",
            "lead_type",
            "lead_behaviour_profile",
            "has_company",
            "has_gtin",
            "average_stock",
        ],
    ),
}

```

```

        "business_type",
        "declared_product_catalog_size",
        "declared_monthly_revenue",
    ],
),
"olist_orders_dataset.csv": (
    "raw.olist_orders",
    [
        "order_id",
        "customer_id",
        "order_status",
        "order_purchase_timestamp",
        "order_approved_at",
        "order_delivered_carrier_date",
        "order_delivered_customer_date",
        "order_estimated_delivery_date",
    ],
),
"olist_order_items_dataset.csv": (
    "raw.olist_order_items",
    [
        "order_id",
        "order_item_id",
        "product_id",
        "seller_id",
        "shipping_limit_date",
        "price",
        "freight_value",
    ],
),
"olist_order_payments_dataset.csv": (
    "raw.olist_order_payments",
    [
        "order_id",
        "payment_sequential",
        "payment_type",
        "payment_installments",
        "payment_value",
    ],
),
"olist_order_reviews_dataset.csv": (
    "raw.olist_order_reviews",
    [
        "review_id",
        "order_id",
        "review_score",
        "review_comment_title",
        "review_comment_message",
        "review_creation_date",
        "review_answer_timestamp",
    ],
),
"olist_products_dataset.csv": (
    "raw.olist_products",
    [
        "product_id",
        "product_category_name",
        "product_name_lenght",
        "product_description_lenght",
        "product_photos_qty",
        "product_weight_g",
        "product_length_cm",
        "product_height_cm",
        "product_width_cm",
    ],
),
"olist_sellers_dataset.csv": (
    "raw.olist_sellers",
    ["seller_id", "seller_zip_code_prefix", "seller_city", "seller_state"],
),
"product_category_name_translation.csv": (
    "raw.product_category_name_translation",
    ["product_category_name", "product_category_name_english"],
),
}

```

```

RAW_TRUNCATE_ORDER = [table for table, _ in RAW_TABLES.values()]

```

```

def truncate_raw(engine: Engine) -> None:
    logger.info("Truncating raw tables")
    with engine.begin() as conn:
        for table in RAW_TRUNCATE_ORDER:
            conn.execute(text(f"TRUNCATE TABLE {table} RESTART IDENTITY CASCADE"))

```

```

def copy_csv_to_table(engine: Engine, csv_path: Path, table_name: str, columns: list[str]) -> None:

```

```

if not csv_path.exists():
    raise FileNotFoundError(f"Missing raw CSV: {csv_path}")

column_list = ", ".join(columns)
copy_sql = f"""
    COPY {table_name} ({column_list})
    FROM STDIN
    WITH (FORMAT CSV, HEADER TRUE, NULL '', ENCODING 'UTF8')
"""

logger.info("Loading %s into %s", csv_path.name, table_name)
raw_conn = engine.raw_connection()
try:
    with raw_conn.cursor() as cur, csv_path.open("r", encoding="utf-8") as f:
        cur.copy_expert(copy_sql, f)
    raw_conn.commit()
except Exception:
    raw_conn.rollback()
    raise
finally:
    raw_conn.close()

def load_all_raw(engine: Engine, raw_data_dir: Path | None = None) -> None:
    raw_data_dir = raw_data_dir or settings.raw_data_dir
    truncate_raw(engine)
    for filename, (table_name, columns) in RAW_TABLES.items():
        copy_csv_to_table(engine, raw_data_dir / filename, table_name, columns)

```

src/load_dwh.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/src/load_dwh.py

```

FILE: src/load_dwh.py
```python
from __future__ import annotations

import logging

from sqlalchemy.engine import Engine

from .db import execute_many

logger = logging.getLogger(__name__)

TRUNCATE_DWH_SQL = [
 """
 TRUNCATE TABLE
 dwh.fact_reviews,
 dwh.fact_payments,
 dwh.fact_order_items,
 dwh.fact_orders,
 dwh.fact_closed_deals,
 dwh.fact_marketing_leads,
 dwh.dim_sales_rep,
 dwh.dim_lead_profile,
 dwh.dim_lead_type,
 dwh.dim_business_segment,
 dwh.dim_landing_page,
 dwh.dim_marketing_origin,
 dwh.dim_payment_type,
 dwh.dim_order_status,
 dwh.dim_product,
 dwh.dim_product_category,
 dwh.dim_seller,
 dwh.dim_customer,
 dwh.dim_zip_prefix,
 dwh.dim_city,
 dwh.dim_state,
 dwh.dim_date
 RESTART IDENTITY CASCADE
 """
]

DWH_SQL = [
 # Date dimension
 """
 INSERT INTO dwh.dim_date (
 date_key, full_date, day_of_month, day_of_week, day_name,
 week_of_year, month_number, month_name, quarter_number, year_number, is_weekend
)
 WITH all_dates AS (

```

```

SELECT first_contact_date::date AS dt FROM staging.stg_marketing_leads
UNION SELECT won_ts::date FROM staging.stg_closed_deals
UNION SELECT order_purchase_ts::date FROM staging.stg_orders
UNION SELECT order_approved_ts::date FROM staging.stg_orders
UNION SELECT order_delivered_carrier_ts::date FROM staging.stg_orders
UNION SELECT order_delivered_customer_ts::date FROM staging.stg_orders
UNION SELECT order_estimated_delivery_ts::date FROM staging.stg_orders
UNION SELECT shipping_limit_ts::date FROM staging.stg_order_items
UNION SELECT review_creation_ts::date FROM staging.stg_reviews
UNION SELECT review_answer_ts::date FROM staging.stg_reviews
), bounds AS (
 SELECT MIN(dt) AS min_dt, MAX(dt) AS max_dt
 FROM all_dates
 WHERE dt IS NOT NULL
)
SELECT
 TO_CHAR(d::date, 'YYYYMMDD')::integer AS date_key,
 d::date AS full_date,
 EXTRACT(DAY FROM d)::smallint AS day_of_month,
 EXTRACT(ISODOW FROM d)::smallint AS day_of_week,
 TRIM(TO_CHAR(d, 'Day')) AS day_name,
 EXTRACT(WEEK FROM d)::smallint AS week_of_year,
 EXTRACT(MONTH FROM d)::smallint AS month_number,
 TRIM(TO_CHAR(d, 'Month')) AS month_name,
 EXTRACT(QUARTER FROM d)::smallint AS quarter_number,
 EXTRACT(YEAR FROM d)::integer AS year_number,
 EXTRACT(ISODOW FROM d) IN (6, 7) AS is_weekend
FROM bounds b,
 GENERATE_SERIES(b.min_dt, b.max_dt, interval '1 day') AS d
""",
State dimension
""",
INSERT INTO dwh.dim_state (state_code, state_name)
WITH states AS (
 SELECT state_code FROM staging.stg_geolocation
 UNION SELECT state_code FROM staging.stg_customers
 UNION SELECT state_code FROM staging.stg_sellers
)
SELECT DISTINCT
 state_code,
 CASE state_code
 WHEN 'AC' THEN 'Acre'
 WHEN 'AL' THEN 'Alagoas'
 WHEN 'AP' THEN 'Amapá'
 WHEN 'AM' THEN 'Amazonas'
 WHEN 'BA' THEN 'Bahia'
 WHEN 'CE' THEN 'Ceará'
 WHEN 'DF' THEN 'Distrito Federal'
 WHEN 'ES' THEN 'Espírito Santo'
 WHEN 'GO' THEN 'Goiás'
 WHEN 'MA' THEN 'Maranhão'
 WHEN 'MT' THEN 'Mato Grosso'
 WHEN 'MS' THEN 'Mato Grosso do Sul'
 WHEN 'MG' THEN 'Minas Gerais'
 WHEN 'PA' THEN 'Pará'
 WHEN 'PB' THEN 'Paraíba'
 WHEN 'PR' THEN 'Paraná'
 WHEN 'PE' THEN 'Pernambuco'
 WHEN 'PI' THEN 'Piauí'
 WHEN 'RJ' THEN 'Rio de Janeiro'
 WHEN 'RN' THEN 'Rio Grande do Norte'
 WHEN 'RS' THEN 'Rio Grande do Sul'
 WHEN 'RO' THEN 'Rondônia'
 WHEN 'RR' THEN 'Roraima'
 WHEN 'SC' THEN 'Santa Catarina'
 WHEN 'SP' THEN 'São Paulo'
 WHEN 'SE' THEN 'Sergipe'
 WHEN 'TO' THEN 'Tocantins'
 ELSE NULL
 END AS state_name
FROM states
WHERE state_code IS NOT NULL AND state_code <> ''
""",
City dimension
""",
INSERT INTO dwh.dim_city (city_name, state_key)
WITH cities AS (
 SELECT city_name, state_code FROM staging.stg_geolocation
 UNION SELECT city_name, state_code FROM staging.stg_customers
 UNION SELECT city_name, state_code FROM staging.stg_sellers
)
SELECT DISTINCT
 c.city_name,
 s.state_key
FROM cities c
JOIN dwh.dim_state s ON c.state_code = s.state_code
WHERE c.city_name IS NOT NULL AND c.city_name <> ''

```

```

""",
ZIP prefix dimension
""",
INSERT INTO dwh.dim_zip_prefix (zip_prefix, city_key, state_key, latitude, longitude)
WITH all_zips AS (
 SELECT zip_prefix, city_name, state_code, latitude, longitude, 1 AS priority
 FROM staging.stg_geolocation
 UNION ALL
 SELECT zip_prefix, city_name, state_code, NULL::numeric AS latitude, NULL::numeric AS longitude, 2 AS priority
 FROM staging.stg_customers
 UNION ALL
 SELECT zip_prefix, city_name, state_code, NULL::numeric AS latitude, NULL::numeric AS longitude, 3 AS priority
 FROM staging.stg_sellers
), ranked AS (
 SELECT
 zip_prefix,
 city_name,
 state_code,
 latitude,
 longitude,
 ROW_NUMBER() OVER (
 PARTITION BY zip_prefix
 ORDER BY priority, city_name NULLS LAST, state_code NULLS LAST
) AS rn
 FROM all_zips
 WHERE zip_prefix IS NOT NULL AND zip_prefix <> ''
)
SELECT
 r.zip_prefix,
 c.city_key,
 s.state_key,
 r.latitude,
 r.longitude
FROM ranked r
LEFT JOIN dwh.dim_state s ON r.state_code = s.state_code
LEFT JOIN dwh.dim_city c ON r.city_name = c.city_name AND c.state_key = s.state_key
WHERE r.rn = 1
""",
Customer dimension
""",
INSERT INTO dwh.dim_customer (customer_id, customer_unique_id, zip_prefix_key, city_key, state_key)
SELECT
 c.customer_id,
 c.customer_unique_id,
 z.zip_prefix_key,
 city.city_key,
 state.state_key
FROM staging.stg_customers c
LEFT JOIN dwh.dim_zip_prefix z ON c.zip_prefix = z.zip_prefix
LEFT JOIN dwh.dim_state state ON c.state_code = state.state_code
LEFT JOIN dwh.dim_city city ON c.city_name = city.city_name AND city.state_key = state.state_key
""",
Seller dimension
""",
INSERT INTO dwh.dim_seller (seller_id, zip_prefix_key, city_key, state_key)
SELECT
 s.seller_id,
 z.zip_prefix_key,
 city.city_key,
 state.state_key
FROM staging.stg_sellers s
LEFT JOIN dwh.dim_zip_prefix z ON s.zip_prefix = z.zip_prefix
LEFT JOIN dwh.dim_state state ON s.state_code = state.state_code
LEFT JOIN dwh.dim_city city ON s.city_name = city.city_name AND city.state_key = state.state_key
""",
Product dimensions
""",
INSERT INTO dwh.dim_product_category (product_category_name, product_category_name_english)
SELECT product_category_name, product_category_name_english
FROM staging.stg_product_categories
""",
""",
INSERT INTO dwh.dim_product (
 product_id, product_category_key, product_name_length, product_description_length,
 product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm
)
SELECT
 p.product_id,
 pc.product_category_key,
 p.product_name_length,
 p.product_description_length,
 p.product_photos_qty,
 p.product_weight_g,
 p.product_length_cm,
 p.product_height_cm,
 p.product_width_cm
FROM staging.stg_products p

```

```

LEFT JOIN dwh.dim_product_category pc ON p.product_category_name = pc.product_category_name
''',
Small dimensions
'''
INSERT INTO dwh.dim_order_status (order_status)
SELECT DISTINCT COALESCE(NULLIF(order_status, ''), 'unknown')
FROM staging.stg_orders
''',
'''
INSERT INTO dwh.dim_payment_type (payment_type)
SELECT DISTINCT COALESCE(NULLIF(payment_type, ''), 'unknown')
FROM staging.stg_payments
''',
'''
INSERT INTO dwh.dim_marketing_origin (origin_name)
SELECT DISTINCT COALESCE(NULLIF(origin, ''), 'unknown')
FROM staging.stg_marketing_leads
''',
'''
INSERT INTO dwh.dim_landing_page (landing_page_id)
SELECT DISTINCT COALESCE(NULLIF(landing_page_id, ''), 'unknown')
FROM staging.stg_marketing_leads
''',
'''
INSERT INTO dwh.dim_business_segment (business_segment)
SELECT DISTINCT COALESCE(NULLIF(business_segment, ''), 'unknown')
FROM staging.stg_closed_deals
''',
'''
INSERT INTO dwh.dim_lead_type (lead_type)
SELECT DISTINCT COALESCE(NULLIF(lead_type, ''), 'unknown')
FROM staging.stg_closed_deals
''',
'''
INSERT INTO dwh.dim_lead_profile (lead_behaviour_profile)
SELECT DISTINCT COALESCE(NULLIF(lead_behaviour_profile, ''), 'unknown')
FROM staging.stg_closed_deals
''',
'''
INSERT INTO dwh.dim_sales_rep (sales_rep_id, sales_role)
SELECT DISTINCT sdr_id, 'SDR'
FROM staging.stg_closed_deals
WHERE sdr_id IS NOT NULL AND sdr_id <> ''
UNION
SELECT DISTINCT sr_id, 'SR'
FROM staging.stg_closed_deals
WHERE sr_id IS NOT NULL AND sr_id <> ''
''',
Facts
'''
INSERT INTO dwh.fact_marketing_leads (
 mql_id, first_contact_date_key, landing_page_key, marketing_origin_key, lead_count
)
SELECT
 ml.mql_id,
 TO_CHAR(ml.first_contact_date, 'YYYYMMDD')::integer AS first_contact_date_key,
 lp.landing_page_key,
 mo.marketing_origin_key,
 1 AS lead_count
FROM staging.stg_marketing_leads ml
LEFT JOIN dwh.dim_landing_page lp ON ml.landing_page_id = lp.landing_page_id
LEFT JOIN dwh.dim_marketing_origin mo ON ml.origin = mo.origin_name
''',
'''
INSERT INTO dwh.fact_closed_deals (
 mql_id, seller_key, won_date_key, sdr_sales_rep_key, sr_sales_rep_key,
 business_segment_key, lead_type_key, lead_profile_key,
 has_company, has_gtin, average_stock, business_type,
 declared_product_catalog_size, declared_monthly_revenue,
 is_active_seller, deal_count
)
SELECT
 cd.mql_id,
 ds.seller_key,
 TO_CHAR(cd.won_ts::date, 'YYYYMMDD')::integer AS won_date_key,
 sdr.sales_rep_key AS sdr_sales_rep_key,
 sr.sales_rep_key AS sr_sales_rep_key,
 bs.business_segment_key,
 lt.lead_type_key,
 lp.lead_profile_key,
 cd.has_company,
 cd.has_gtin,
 cd.average_stock,
 cd.business_type,
 cd.declared_product_catalog_size,
 cd.declared_monthly_revenue,
 ds.seller_key IS NOT NULL AS is_active_seller,

```

```

1 AS deal_count
FROM staging.stg_closed_deals cd
LEFT JOIN dwh.dim_seller ds ON cd.seller_id = ds.seller_id
LEFT JOIN dwh.dim_sales_rep sdr ON cd.sdr_id = sdr.sales_rep_id AND sdr.sales_role = 'SDR'
LEFT JOIN dwh.dim_sales_rep sr ON cd.sr_id = sr.sales_rep_id AND sr.sales_role = 'SR'
LEFT JOIN dwh.dim_business_segment bs ON cd.business_segment = bs.business_segment
LEFT JOIN dwh.dim_lead_type lt ON cd.lead_type = lt.lead_type
LEFT JOIN dwh.dim_lead_profile lp ON cd.lead_behaviour_profile = lp.lead_behaviour_profile
""",
""",
INSERT INTO dwh.fact_orders (
 order_id, customer_key, order_status_key,
 purchase_date_key, approved_date_key, delivered_carrier_date_key,
 delivered_customer_date_key, estimated_delivery_date_key,
 order_purchase_ts, order_approved_ts, order_delivered_carrier_ts,
 order_delivered_customer_ts, order_estimated_delivery_ts,
 approval_delay_hours, days_to_carrier, days_to_customer,
 estimated_delivery_days, delivery_delay_days,
 is_delivered_on_time, is_delivered, order_count
)
SELECT
 o.order_id,
 dc.customer_key,
 os.order_status_key,
 TO_CHAR(o.order_purchase_ts::date, 'YYYYMMDD')::integer AS purchase_date_key,
 TO_CHAR(o.order_approved_ts::date, 'YYYYMMDD')::integer AS approved_date_key,
 TO_CHAR(o.order_delivered_carrier_ts::date, 'YYYYMMDD')::integer AS delivered_carrier_date_key,
 TO_CHAR(o.order_delivered_customer_ts::date, 'YYYYMMDD')::integer AS delivered_customer_date_key,
 TO_CHAR(o.order_estimated_delivery_ts::date, 'YYYYMMDD')::integer AS estimated_delivery_date_key,
 o.order_purchase_ts,
 o.order_approved_ts,
 o.order_delivered_carrier_ts,
 o.order_delivered_customer_ts,
 o.order_estimated_delivery_ts,
 ROUND((EXTRACT(EPOCH FROM (o.order_approved_ts - o.order_purchase_ts)) / 3600.0)::numeric, 2) AS approval_delay_hou
 ROUND((EXTRACT(EPOCH FROM (o.order_delivered_carrier_ts - o.order_purchase_ts)) / 86400.0)::numeric, 2) AS days_to_
 ROUND((EXTRACT(EPOCH FROM (o.order_delivered_customer_ts - o.order_purchase_ts)) / 86400.0)::numeric, 2) AS days_to_
 ROUND((EXTRACT(EPOCH FROM (o.order_estimated_delivery_ts - o.order_purchase_ts)) / 86400.0)::numeric, 2) AS estimat
 ROUND((EXTRACT(EPOCH FROM (o.order_delivered_customer_ts - o.order_estimated_delivery_ts)) / 86400.0)::numeric, 2)
CASE
 WHEN o.order_delivered_customer_ts IS NULL OR o.order_estimated_delivery_ts IS NULL THEN NULL
 ELSE o.order_delivered_customer_ts <= o.order_estimated_delivery_ts
END AS is_delivered_on_time,
o.order_delivered_customer_ts IS NOT NULL AS is_delivered,
1 AS order_count
FROM staging.stg_orders o
LEFT JOIN dwh.dim_customer dc ON o.customer_id = dc.customer_id
LEFT JOIN dwh.dim_order_status os ON COALESCE(o.order_status, 'unknown') = os.order_status
""",
""",
INSERT INTO dwh.fact_order_items (
 order_id, order_item_id, order_key, product_key, seller_key,
 shipping_limit_date_key, shipping_limit_ts,
 item_price, freight_value, total_item_value, item_count
)
SELECT
 oi.order_id,
 oi.order_item_id,
 fo.order_key,
 dp.product_key,
 ds.seller_key,
 TO_CHAR(oi.shipping_limit_ts::date, 'YYYYMMDD')::integer AS shipping_limit_date_key,
 oi.shipping_limit_ts,
 oi.price,
 oi.freight_value,
 COALESCE(oi.price, 0) + COALESCE(oi.freight_value, 0) AS total_item_value,
 1 AS item_count
FROM staging.stg_order_items oi
LEFT JOIN dwh.fact_orders fo ON oi.order_id = fo.order_id
LEFT JOIN dwh.dim_product dp ON oi.product_id = dp.product_id
LEFT JOIN dwh.dim_seller ds ON oi.seller_id = ds.seller_id
""",
""",
INSERT INTO dwh.fact_payments (
 order_id, payment_sequential, order_key, payment_type_key,
 payment_installments, payment_value, payment_count
)
SELECT
 p.order_id,
 p.payment_sequential,
 fo.order_key,
 pt.payment_type_key,
 p.payment_installments,
 p.payment_value,
 1 AS payment_count
FROM staging.stg_payments p
LEFT JOIN dwh.fact_orders fo ON p.order_id = fo.order_id

```

```

LEFT JOIN dwh.dim_payment_type pt ON COALESCE(p.payment_type, 'unknown') = pt.payment_type
"""
"""
INSERT INTO dwh.fact_reviews (
 review_id, order_id, order_key,
 review_creation_date_key, review_answer_date_key,
 review_score, review_comment_title, review_comment_message,
 review_creation_ts, review_answer_ts, review_response_time_hours,
 has_review_title, has_review_message,
 is_positive_review, is_neutral_review, is_negative_review, review_count
)
SELECT
 r.review_id,
 r.order_id,
 fo.order_key,
 TO_CHAR(r.review_creation_ts::date, 'YYYYMMDD')::integer AS review_creation_date_key,
 TO_CHAR(r.review_answer_ts::date, 'YYYYMMDD')::integer AS review_answer_date_key,
 r.review_score,
 r.review_comment_title,
 r.review_comment_message,
 r.review_creation_ts,
 r.review_answer_ts,
 ROUND((EXTRACT(EPOCH FROM (r.review_answer_ts - r.review_creation_ts)) / 3600.0)::numeric, 2) AS review_response_time_hours,
 NULLIF(TRIM(COALESCE(r.review_comment_title, '')), '') IS NOT NULL AS has_review_title,
 NULLIF(TRIM(COALESCE(r.review_comment_message, '')), '') IS NOT NULL AS has_review_message,
 r.review_score >= 4 AS is_positive_review,
 r.review_score = 3 AS is_neutral_review,
 r.review_score <= 2 AS is_negative_review,
 1 AS review_count
FROM staging.stg_reviews r
LEFT JOIN dwh.fact_orders fo ON r.order_id = fo.order_id
"""
]

def load_dwh(engine: Engine) -> None:
 logger.info("Loading DWH dimensions and facts")
 execute_many(engine, TRUNCATE_DWH_SQL + DWH_SQL)
...

```

## src/quality\_checks.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/src/quality\_checks.py

```

FILE: src/quality_checks.py
```python
from __future__ import annotations

import logging
from dataclasses import dataclass
from typing import Any

from sqlalchemy import text
from sqlalchemy.engine import Engine

logger = logging.getLogger(__name__)

@dataclass
class QualityCheckResult:
    name: str
    value: Any
    passed: bool
    expectation: str

CHECKS = [
    (
        "raw_orders_loaded",
        "SELECT COUNT(*) FROM raw.olist_orders",
        lambda x: x > 0,
        "> 0 rows",
    ),
    (
        "dwh_fact_orders_loaded",
        "SELECT COUNT(*) FROM dwh.fact_orders",
        lambda x: x > 0,
        "> 0 rows",
    ),
    (
        "fact_order_items_missing_order_key",
        "SELECT COUNT(*) FROM dwh.fact_order_items WHERE order_key IS NULL",
        lambda x: x == 0,
    ),
]

```

```

        "0 missing order_key",
    ),
    (
        "fact_order_items_missing_product_key",
        "SELECT COUNT(*) FROM dwh.fact_order_items WHERE product_key IS NULL",
        lambda x: x == 0,
        "0 missing product_key",
    ),
    (
        "fact_order_items_missing_seller_key",
        "SELECT COUNT(*) FROM dwh.fact_order_items WHERE seller_key IS NULL",
        lambda x: x == 0,
        "0 missing seller_key",
    ),
    (
        "fact_payments_missing_order_key",
        "SELECT COUNT(*) FROM dwh.fact_payments WHERE order_key IS NULL",
        lambda x: x == 0,
        "0 missing order_key",
    ),
    (
        "fact_reviews_missing_order_key",
        "SELECT COUNT(*) FROM dwh.fact_reviews WHERE order_key IS NULL",
        lambda x: x == 0,
        "0 missing order_key",
    ),
    (
        "reviews_outside_1_5",
        "SELECT COUNT(*) FROM dwh.fact_reviews WHERE review_score NOT BETWEEN 1 AND 5",
        lambda x: x == 0,
        "0 reviews outside 1..5",
    ),
]

def run_quality_checks(engine: Engine, fail_fast: bool = False) -> list[QualityCheckResult]:
    results: list[QualityCheckResult] = []
    with engine.connect() as conn:
        for name, sql, predicate, expectation in CHECKS:
            value = conn.execute(text(sql)).scalar()
            passed = bool(predicate(value))
            result = QualityCheckResult(name, value, passed, expectation)
            results.append(result)
            status = "PASS" if passed else "FAIL"
            logger.info("%s | %s | value=%s | expected=%s", status, name, value, expectation)
            if fail_fast and not passed:
                raise RuntimeError(f"Quality check failed: {name}. Value={value}, expected={expectation}")
    return results

def print_quality_report(results: list[QualityCheckResult]) -> None:
    print("\nQuality check report")
    print("-" * 80)
    for r in results:
        status = "PASS" if r.passed else "FAIL"
        print(f"{status:4} | {r.name:40} | value={r.value!s:10} | expected={r.expectation}")
    print("-" * 80)
    failed = [r for r in results if not r.passed]
    print(f"Total checks: {len(results)} | Failed: {len(failed)}")

```

src/run_etl.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/src/run_etl.py

```

FILE: src/run_etl.py
```python
from __future__ import annotations

import argparse
import logging
from pathlib import Path
from typing import NoReturn

from sqlalchemy.exc import OperationalError

from .config import settings
from .db import execute_sql_file, get_engine
from .extract_load_raw import load_all_raw
from .load_dwh import load_dwh
from .quality_checks import print_quality_report, run_quality_checks
from .transform_staging import build_staging

```

```

logging.basicConfig(
 level=logging.INFO,
 format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
)
logger = logging.getLogger(__name__)

def _raise_database_unavailable(exc: OperationalError) -> NoReturn:
 logger.error(
 "Could not connect to PostgreSQL. Start the database first and verify "
 "DATABASE_URL. Current value: %s",
 settings.database_url,
)
 raise SystemExit(1) from exc

def create_schema() -> None:
 engine = get_engine()
 execute_sql_file(engine, settings.schema_sql_path)
 logger.info("Schemas and tables are ready")

def create_marts() -> None:
 engine = get_engine()
 execute_sql_file(engine, settings.marts_sql_path)
 logger.info("Marts are ready")

def run_all(raw_data_dir: Path | None = None, fail_fast_quality: bool = False) -> None:
 engine = get_engine()
 logger.info("Starting full Olist ELT pipeline")

 execute_sql_file(engine, settings.schema_sql_path)
 load_all_raw(engine, raw_data_dir or settings.raw_data_dir)
 build_staging(engine)
 load_dwh(engine)
 execute_sql_file(engine, settings.marts_sql_path)
 results = run_quality_checks(engine, fail_fast=fail_fast_quality)
 print_quality_report(results)

 logger.info("Pipeline completed")

def main() -> None:
 parser = argparse.ArgumentParser(description="Run the Olist PostgreSQL ELT pipeline.")
 parser.add_argument(
 "--step",
 choices=["schema", "raw", "staging", "dwh", "marts", "quality", "all"],
 default="all",
 help="Pipeline step to run.",
)
 parser.add_argument(
 "--raw-data-dir",
 type=Path,
 default=settings.raw_data_dir,
 help="Path to the folder containing raw CSV files.",
)
 parser.add_argument(
 "--fail-fast-quality",
 action="store_true",
 help="Raise an error if a quality check fails.",
)
 args = parser.parse_args()

 engine = get_engine()

 if args.step == "schema":
 execute_sql_file(engine, settings.schema_sql_path)
 elif args.step == "raw":
 load_all_raw(engine, args.raw_data_dir)
 elif args.step == "staging":
 build_staging(engine)
 elif args.step == "dwh":
 load_dwh(engine)
 elif args.step == "marts":
 execute_sql_file(engine, settings.marts_sql_path)
 elif args.step == "quality":
 results = run_quality_checks(engine, fail_fast=args.fail_fast_quality)
 print_quality_report(results)
 elif args.step == "all":
 run_all(args.raw_data_dir, args.fail_fast_quality)

if __name__ == "__main__":
 try:
 main()
 except OperationalError as exc:

```

```
 _raise_database_unavailable(exc)
 ...
```

## src/transform\_staging.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/src/transform\_staging.py

```
FILE: src/transform_staging.py
```python
from __future__ import annotations

import logging

from sqlalchemy.engine import Engine

from .db import execute_many

logger = logging.getLogger(__name__)

STAGING_TABLES = [
    "staging.stg_reviews",
    "staging.stg_payments",
    "staging.stg_order_items",
    "staging.stg_orders",
    "staging.stg_products",
    "staging.stg_product_categories",
    "staging.stg_geolocation",
    "staging.stg_sellers",
    "staging.stg_customers",
    "staging.stg_closed_deals",
    "staging.stg_marketing_leads",
]

TRUNCATE_STAGING_SQL = [
    f"TRUNCATE TABLE {table} RESTART IDENTITY CASCADE" for table in STAGING_TABLES
]

STAGING_SQL = [
    """
    INSERT INTO staging.stg_customers (
        customer_id, customer_unique_id, zip_prefix, city_name, state_code
    )
    SELECT
        customer_id,
        customer_unique_id,
        LPAD(TRIM(customer_zip_code_prefix), 5, '0') AS zip_prefix,
        LOWER(TRIM(customer_city)) AS city_name,
        UPPER(TRIM(customer_state)) AS state_code
    FROM raw.olist_customers
    """,
    """
    INSERT INTO staging.stg_sellers (
        seller_id, zip_prefix, city_name, state_code
    )
    SELECT
        seller_id,
        LPAD(TRIM(seller_zip_code_prefix), 5, '0') AS zip_prefix,
        LOWER(TRIM(seller_city)) AS city_name,
        UPPER(TRIM(seller_state)) AS state_code
    FROM raw.olist_sellers
    """,
    """
    INSERT INTO staging.stg_geolocation (
        zip_prefix, latitude, longitude, city_name, state_code
    )
    WITH cleaned AS (
        SELECT
            LPAD(TRIM(geolocation_zip_code_prefix), 5, '0') AS zip_prefix,
            geolocation_lat AS latitude,
            geolocation_lng AS longitude,
            LOWER(TRIM(geolocation_city)) AS city_name,
            UPPER(TRIM(geolocation_state)) AS state_code
        FROM raw.olist_geolocation
        WHERE geolocation_zip_code_prefix IS NOT NULL
    ),
    geo_avg AS (
        SELECT
            zip_prefix,
            AVG(latitude) AS latitude,
            AVG(longitude) AS longitude
        FROM cleaned
        GROUP BY zip_prefix
    ),
    """
]
```

```

city_rank AS (
    SELECT
        zip_prefix,
        city_name,
        state_code,
        ROW_NUMBER() OVER (
            PARTITION BY zip_prefix
            ORDER BY COUNT(*) DESC, city_name, state_code
        ) AS rn
    FROM cleaned
    GROUP BY zip_prefix, city_name, state_code
)
SELECT
    a.zip_prefix,
    a.latitude,
    a.longitude,
    r.city_name,
    r.state_code
FROM geo_avg a
LEFT JOIN city_rank r
    ON a.zip_prefix = r.zip_prefix
    AND r.rn = 1
""",
""",
INSERT INTO staging.stg_product_categories (
    product_category_name, product_category_name_english
)
WITH source_categories AS (
    SELECT DISTINCT COALESCE(NULLIF(TRIM(product_category_name), ''), 'unknown') AS product_category_name
    FROM raw.olist_products
    UNION
    SELECT DISTINCT COALESCE(NULLIF(TRIM(product_category_name), ''), 'unknown') AS product_category_name
    FROM raw.product_category_name_translation
)
SELECT
    sc.product_category_name,
    COALESCE(t.product_category_name_english, sc.product_category_name) AS product_category_name_english
FROM source_categories sc
LEFT JOIN raw.product_category_name_translation t
    ON sc.product_category_name = t.product_category_name
""",
""",
INSERT INTO staging.stg_products (
    product_id,
    product_category_name,
    product_name_length,
    product_description_length,
    product_photos_qty,
    product_weight_g,
    product_length_cm,
    product_height_cm,
    product_width_cm
)
SELECT
    product_id,
    COALESCE(NULLIF(TRIM(product_category_name), ''), 'unknown') AS product_category_name,
    product_name_lenght AS product_name_length,
    product_description_lenght AS product_description_length,
    product_photos_qty,
    product_weight_g,
    product_length_cm,
    product_height_cm,
    product_width_cm
FROM raw.olist_products
""",
""",
INSERT INTO staging.stg_orders (
    order_id,
    customer_id,
    order_status,
    order_purchase_ts,
    order_approved_ts,
    order_delivered_carrier_ts,
    order_delivered_customer_ts,
    order_estimated_delivery_ts
)
SELECT
    order_id,
    customer_id,
    LOWER(TRIM(order_status)) AS order_status,
    order_purchase_timestamp,
    order_approved_at,
    order_delivered_carrier_date,
    order_delivered_customer_date,
    order_estimated_delivery_date
FROM raw.olist_orders
""",

```

```

""
INSERT INTO staging.stg_order_items (
    order_id,
    order_item_id,
    product_id,
    seller_id,
    shipping_limit_ts,
    price,
    freight_value
)
SELECT
    order_id,
    order_item_id,
    product_id,
    seller_id,
    shipping_limit_date,
    price,
    freight_value
FROM raw.olist_order_items
""
""
INSERT INTO staging.stg_payments (
    order_id,
    payment_sequential,
    payment_type,
    payment_installments,
    payment_value
)
SELECT
    order_id,
    payment_sequential,
    LOWER(TRIM(payment_type)) AS payment_type,
    payment_installments,
    payment_value
FROM raw.olist_order_payments
""
""
INSERT INTO staging.stg_reviews (
    review_id,
    order_id,
    review_score,
    review_comment_title,
    review_comment_message,
    review_creation_ts,
    review_answer_ts
)
SELECT
    review_id,
    order_id,
    review_score,
    review_comment_title,
    review_comment_message,
    review_creation_date,
    review_answer_timestamp
FROM raw.olist_order_reviews
""
""
INSERT INTO staging.stg_marketing_leads (
    mql_id,
    first_contact_date,
    landing_page_id,
    origin
)
SELECT
    mql_id,
    first_contact_date,
    COALESCE(NULLIF(TRIM(landing_page_id), ''), 'unknown') AS landing_page_id,
    COALESCE(NULLIF(LOWER(TRIM(origin)), ''), 'unknown') AS origin
FROM raw.olist_marketing_qualified_leads
""
""
INSERT INTO staging.stg_closed_deals (
    mql_id,
    seller_id,
    sdr_id,
    sr_id,
    won_ts,
    business_segment,
    lead_type,
    lead_behaviour_profile,
    has_company,
    has_gtin,
    average_stock,
    business_type,
    declared_product_catalog_size,
    declared_monthly_revenue
)

```

```

SELECT
    mql_id,
    NULLIF(TRIM(seller_id), '') AS seller_id,
    NULLIF(TRIM(sdr_id), '') AS sdr_id,
    NULLIF(TRIM(sr_id), '') AS sr_id,
    won_date,
    COALESCE(NULLIF(LOWER(TRIM(business_segment)), ''), 'unknown') AS business_segment,
    COALESCE(NULLIF(LOWER(TRIM(lead_type)), ''), 'unknown') AS lead_type,
    COALESCE(NULLIF(LOWER(TRIM(lead_behaviour_profile)), ''), 'unknown') AS lead_behaviour_profile,
    CASE
        WHEN LOWER(TRIM(has_company)) IN ('true', '1', 'yes', 'y', 'sim') THEN TRUE
        WHEN LOWER(TRIM(has_company)) IN ('false', '0', 'no', 'n', 'nao', 'não') THEN FALSE
        ELSE NULL
    END AS has_company,
    CASE
        WHEN LOWER(TRIM(has_gtin)) IN ('true', '1', 'yes', 'y', 'sim') THEN TRUE
        WHEN LOWER(TRIM(has_gtin)) IN ('false', '0', 'no', 'n', 'nao', 'não') THEN FALSE
        ELSE NULL
    END AS has_gtin,
    NULLIF(LOWER(TRIM(average_stock)), '') AS average_stock,
    COALESCE(NULLIF(LOWER(TRIM(business_type)), ''), 'unknown') AS business_type,
    declared_product_catalog_size,
    declared_monthly_revenue
FROM raw.olist_closed_deals
""",
]

def build_staging(engine: Engine) -> None:
    logger.info("Building staging layer")
    execute_many(engine, TRUNCATE_STAGING_SQL + STAGING_SQL)
...

```

tests/test_config_paths.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/tests/test_config_paths.py

```

FILE: tests/test_config_paths.py
```python
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.config import settings

def test_schema_sql_path_prefers_dedicated_schema_file():
 assert settings.schema_sql_path.name == "00_create_schemas_and_tables.sql"

def test_marts_sql_path_prefers_dedicated_marts_file():
 assert settings.marts_sql_path.name == "01_create_marts.sql"

def test_quality_sql_path_points_to_quality_checks_file():
 assert settings.quality_sql_path.name == "99_quality_checks.sql"
 assert settings.quality_sql_path == ROOT / "sql" / "99_quality_checks.sql"
...

```

## tests/test\_dashboard\_app.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/tests/test\_dashboard\_app.py

```

FILE: tests/test_dashboard_app.py
```python
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
APP_PATH = ROOT / "dashboard" / "app.py"

def test_dashboard_app_exists():
    assert APP_PATH.exists()

def test_dashboard_app_has_reusable_helpers():

```

```

source = APP_PATH.read_text(encoding="utf-8")
for helper in [
    "page_header",
    "metric_grid",
    "section_title",
    "standard_chart_layout",
]:
    assert f"def {helper}" in source

def test_dashboard_app_has_navigation_mapping_and_apply_button():
    source = APP_PATH.read_text(encoding="utf-8")
    assert "NAV_OPTIONS = {" in source
    for label in [
        "Executive · Ventes",
        "Commerce · Paiements",
        "Commerce · Satisfaction",
        "Operations · Logistique",
        "Growth · Marketing",
        "Data Quality · Contrôles",
    ]:
        assert label in source
    assert 'selectbox("Navigation"' in source
    assert "Afficher la page" not in source
    assert "st.rerun" not in source
    assert "active_page" not in source
    assert "pending_page_label" not in source

def test_sales_page_does_not_use_pie_chart():
    source = APP_PATH.read_text(encoding="utf-8")
    sales_start = source.index("def page_sales")
    payments_start = source.index("def page_payments")
    sales_source = source[sales_start:payments_start]
    assert "px.pie" not in sales_source

```

tests/test_project_structure.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/tests/test_project_structure.py

```

FILE: tests/test_project_structure.py
```python
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]

def test_dashboard_entrypoint_exists():
 assert (ROOT / "dashboard" / "app.py").exists()

def test_required_sql_files_exist():
 assert (ROOT / "sql" / "01_create_marts.sql").exists()
 assert (ROOT / "sql" / "99_quality_checks.sql").exists()

def test_requirements_include_dashboard_dependencies():
 requirements = (ROOT / "requirements.txt").read_text(encoding="utf-8")
 assert "streamlit" in requirements.lower()
 assert "plotly" in requirements.lower()

def test_raw_data_readme_exists():
 assert (ROOT / "data" / "raw" / "README.md").exists()

```

## tests/test\_sql\_marts.py

Type: text | Source: /mnt/e/projects/olist-data-warehouse-bi/tests/test\_sql\_marts.py

```

FILE: tests/test_sql_marts.py
```python
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
SQL = (ROOT / "sql" / "01_create_marts.sql").read_text(encoding="utf-8").lower()

def test_marts_views_are_defined():

```

```
expected_views = [
    "marts.sales_overview",
    "marts.sales_by_category",
    "marts.payment_analysis",
    "marts.customer_satisfaction",
    "marts.delivery_performance",
    "marts.marketing_funnel",
]
for view in expected_views:
    assert view in SQL

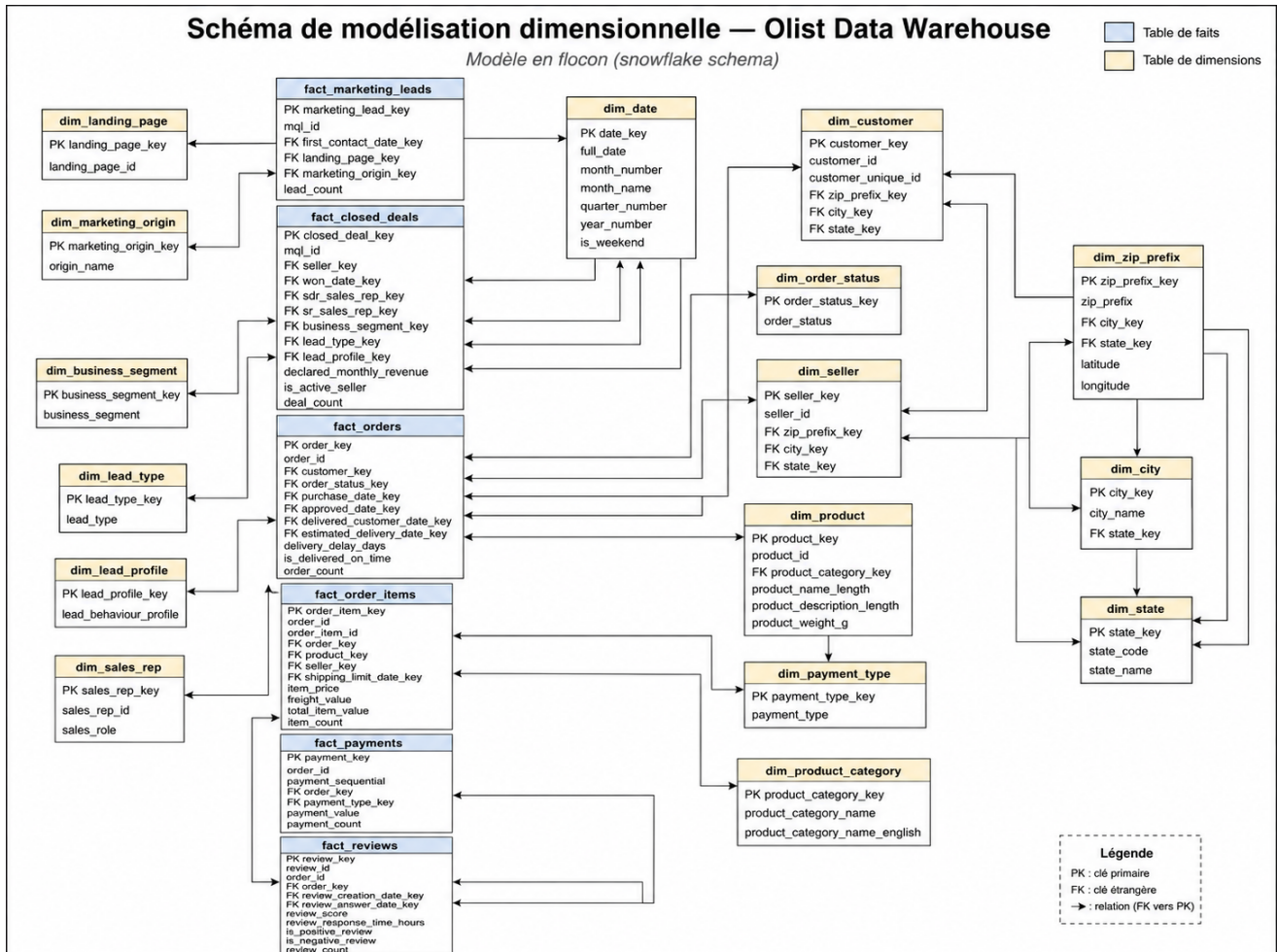
def test_sql_uses_create_or_replace_view():
    assert "create or replace view" in SQL
...
```

3. Images

docs/images/modelisation_dimensionnelle_olist.png

Type: image | Source: /mnt/e/projects/olist-data-warehouse-bi/docs/images/modelisation_dimensionnelle_olist.png

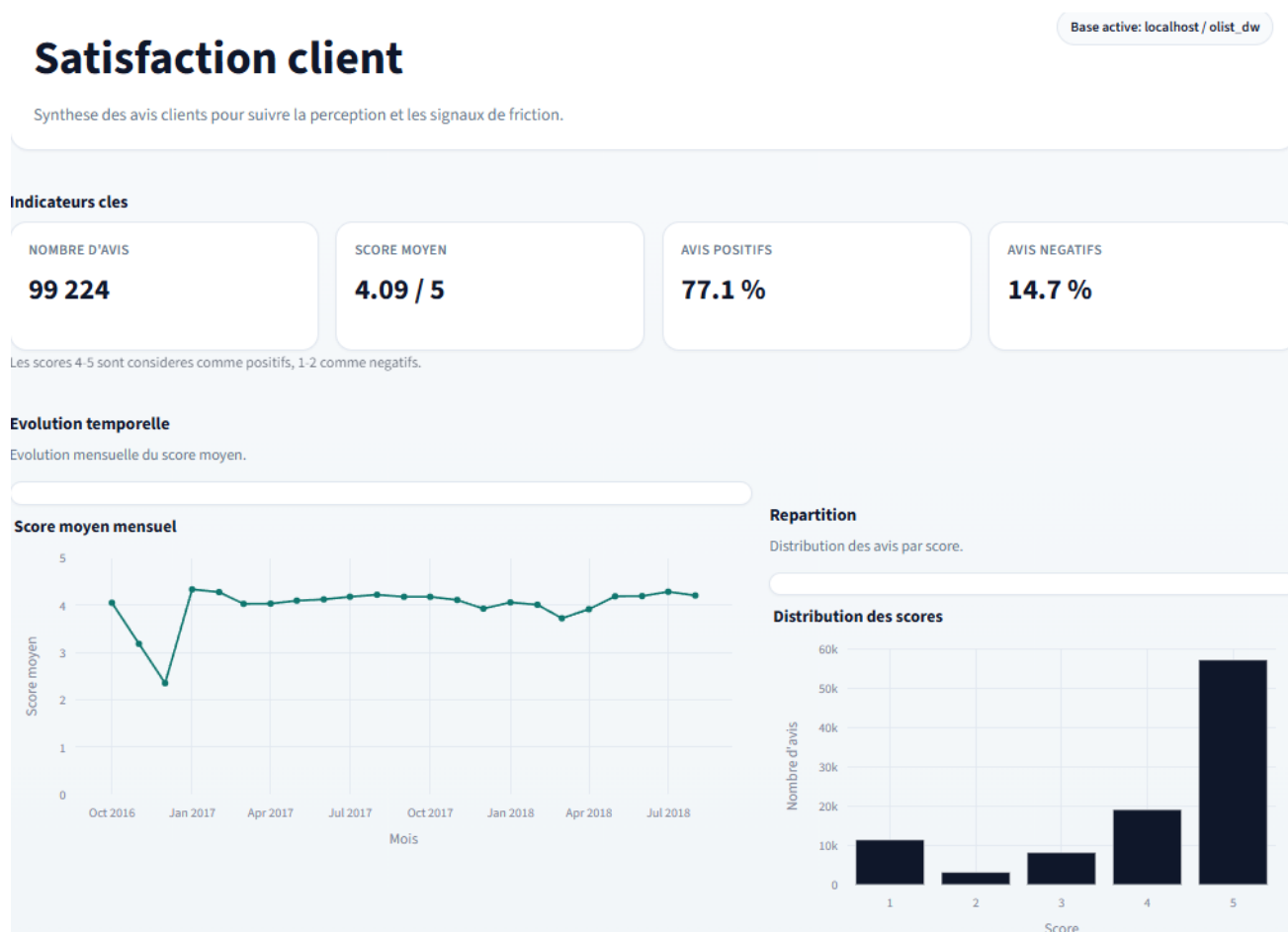
FILE: docs/images/modelisation_dimensionnelle_olist.png (embedded preview)



docs/images/dashboard/dashboard_customer_satisfaction.png

Type: image | Source: /mnt/e/projects/olist-data-warehouse-bi/docs/images/dashboard/dashboard_customer_satisfaction.png

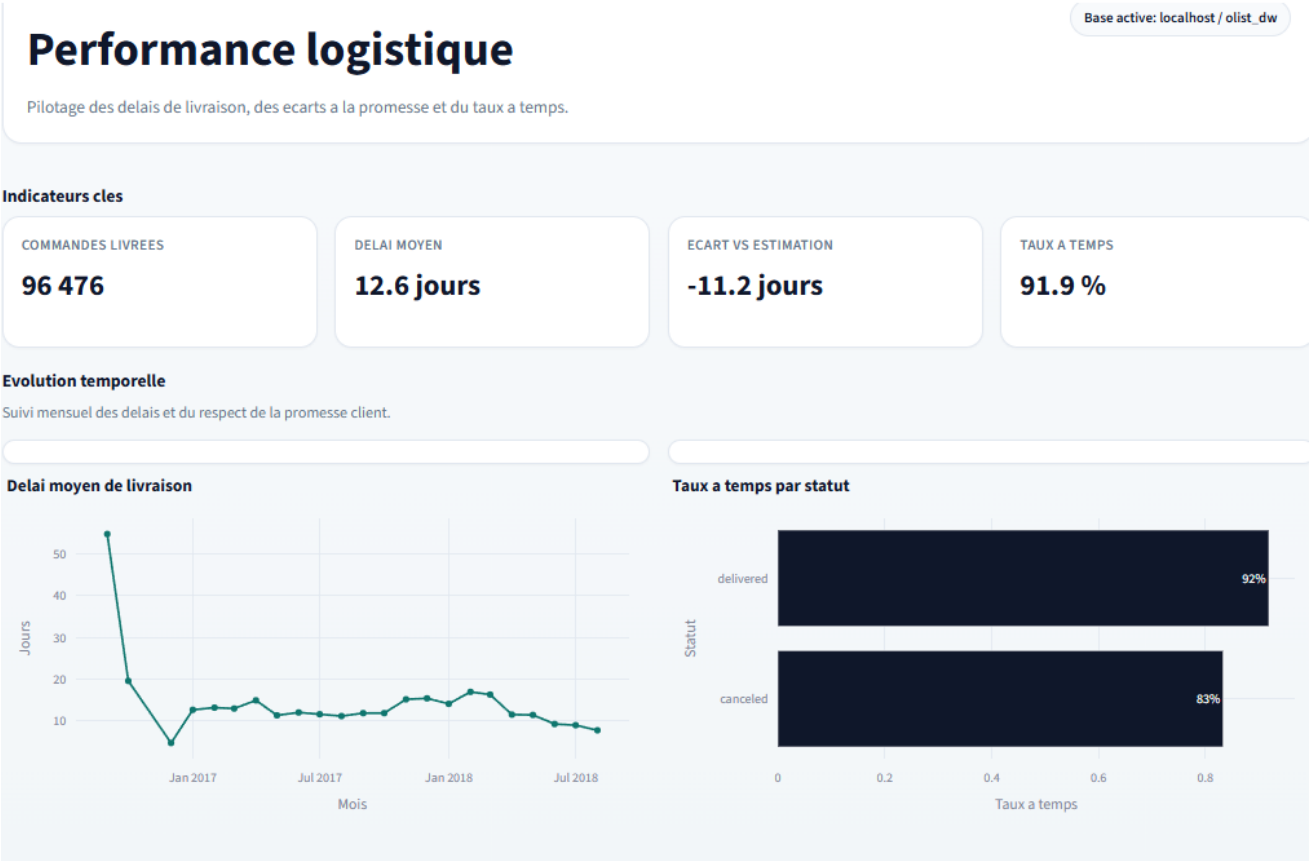
FILE: docs/images/dashboard/dashboard_customer_satisfaction.png (embedded preview)



docs/images/dashboard/dashboard_delivery_performance.png

Type: image | Source: /mnt/e/projects/olist-data-warehouse-bi/docs/images/dashboard/dashboard_delivery_performance.png

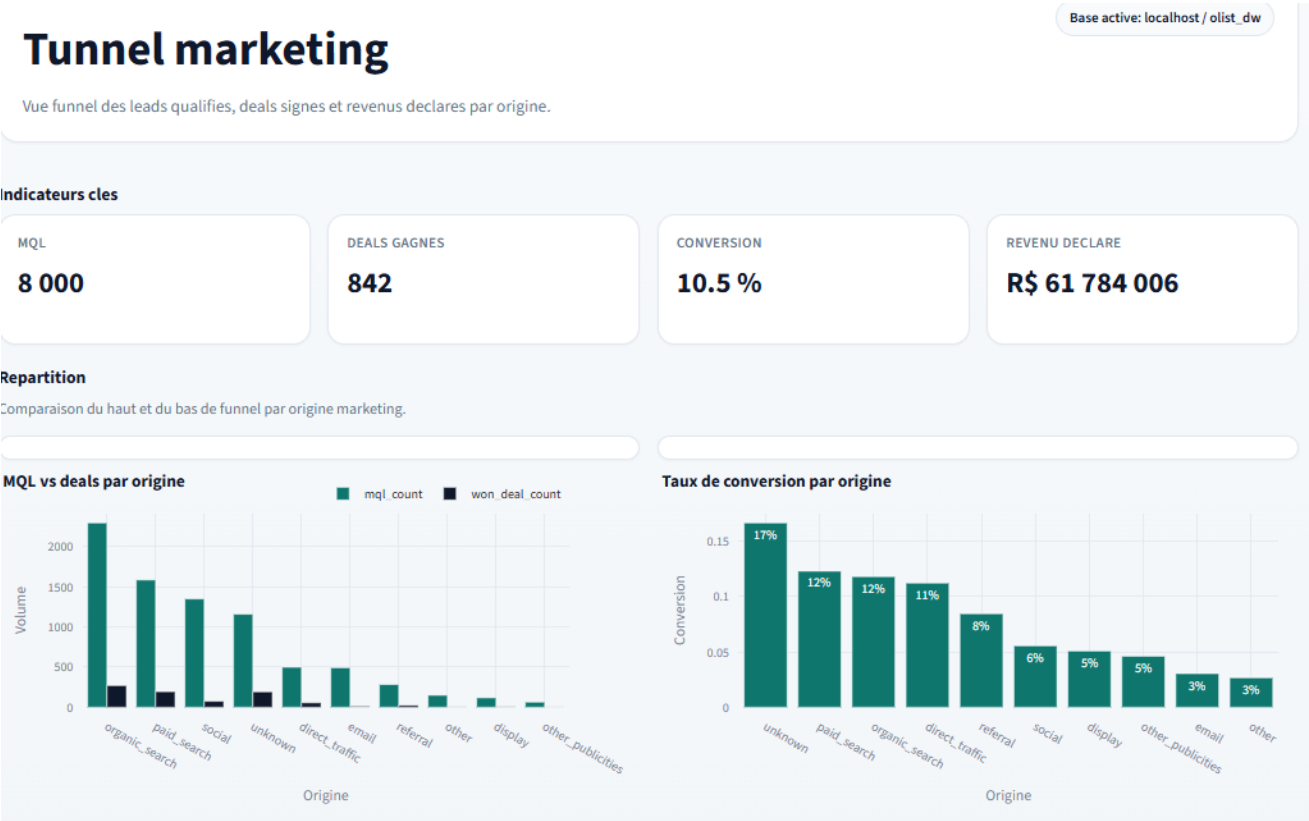
FILE: docs/images/dashboard/dashboard_delivery_performance.png (embedded preview)



docs/images/dashboard/dashboard_marketing_funnel.png

Type: image | Source: /mnt/e/projects/olist-data-warehouse-bi/docs/images/dashboard/dashboard_marketing_funnel.png

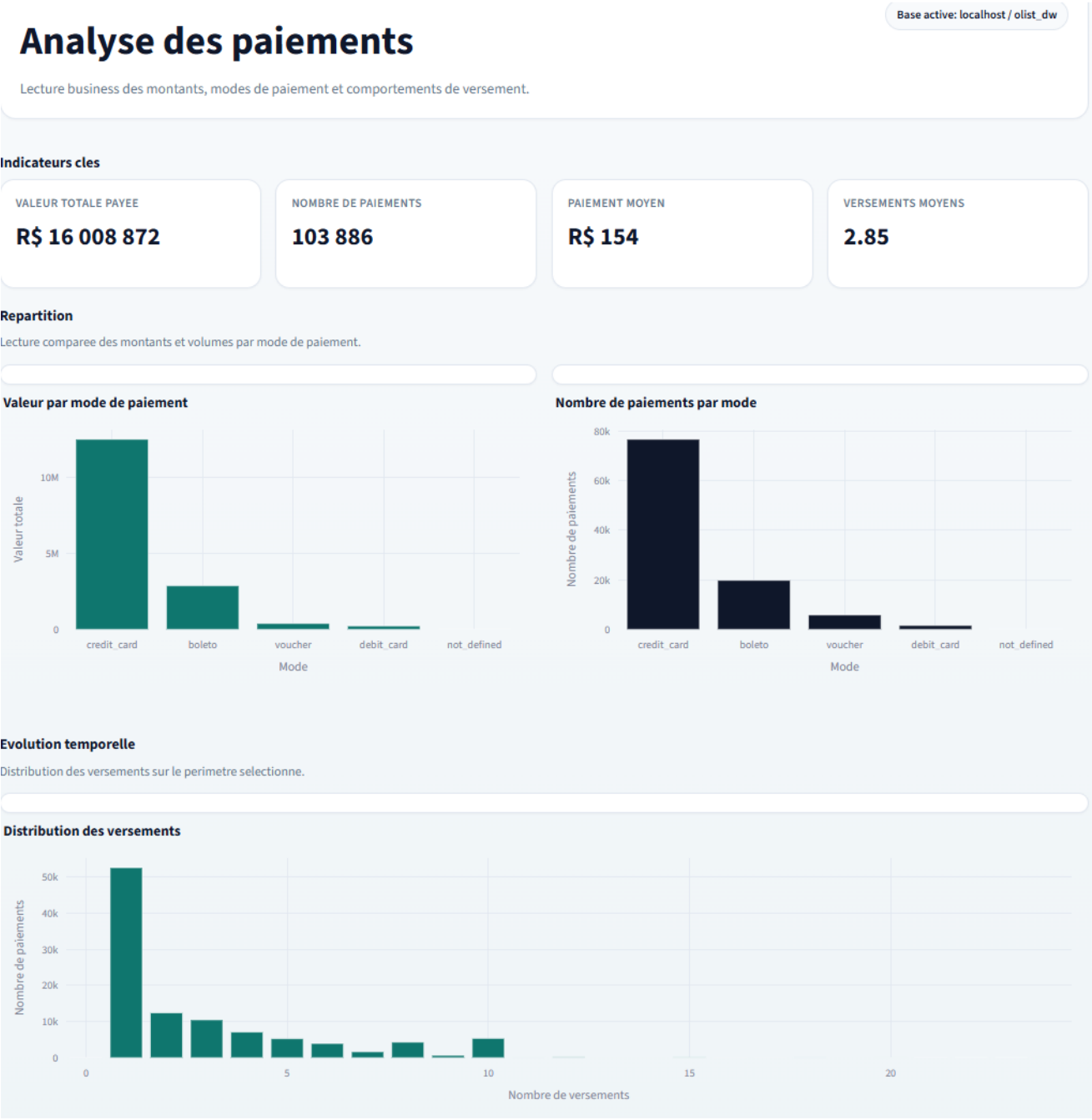
FILE: docs/images/dashboard/dashboard_marketing_funnel.png (embedded preview)



docs/images/dashboard/dashboard_payment_analysis.png

Type: image | Source: /mnt/e/projects/olist-data-warehouse-bi/docs/images/dashboard/dashboard_payment_analysis.png

FILE: docs/images/dashboard/dashboard_payment_analysis.png (embedded preview)



Vue globale des ventes

Suivi executive des revenus, commandes, statuts et categories prioritaires.

Indicateurs clés

REVENU BRUT

R\$ 15 843 553

Somme du revenu brut observe dans marts.sales_overview.

COMMANDES

99 441

Volume total de commandes sur la selection active.

ARTICLES VENDUS

112 650

Nombre d'articles vendus dans les commandes filtres.

PANIER MOYEN

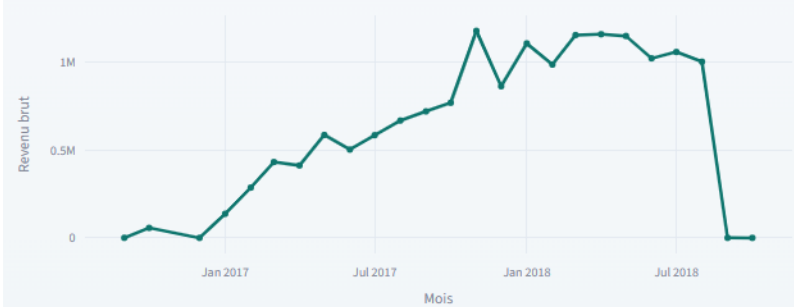
R\$ 159

Revenu brut moyen par commande.

Evolution temporelle

Lecture mensuelle des revenus generes.

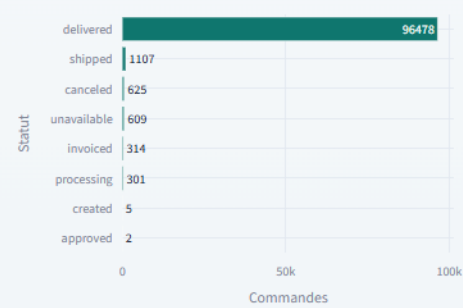
Revenu mensuel



Repartition

Distribution des commandes par statut.

Commandes par statut



Top categories

Top 15 categories produit par revenu brut.

Top categories par revenu brut



4. Skipped Binary Files

- olist_dw_documentation_youssef_khaloufi.docx [application/vnd.openxmlformats-officedocument.wordprocessingml.document]
- olist_dw_documentation_youssef_khaloufi.pdf [application/pdf]

5. Export Summary

- Included files count: 34
- Skipped files count: 142
- Skipped binary files: 2
- Skipped large files: 0
- Warning: no image embed failures